



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр 806

Группа М8О-408Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Интеллектуальный агент для подготовки гражданско-правовых договоров

Автор ВКРБ: Цирулев Николай Владимирович ()

Руководитель: Судаков Владимир Анатольевич ()

Консультант: ()

Консультант: ()

Рецензент: ()

К защите допустить

Заведующий кафедрой № 806 Крылов Сергей Сергеевич ()

____ мая 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 72 страниц, 15 рисунков, 8 таблиц, 25 использованных источников, 4 приложений.

ИНТЕЛЛЕКТУАЛЬНЫЙ АГЕНТ, БОЛЬШИЕ ЯЗЫКОВЫЕ МОДЕЛИ, RETRIEVAL-AUGMENTED GENERATION, LEGAL-TECH, ГЕНЕРАЦИЯ ДОГОВОРОВ, ВЕКТОРНЫЙ ПОИСК

Объектом разработки является интеллектуальный программный агент, который по произвольному словесному описанию сделки готовит проект договора в формате DOCX и сопровождающие юридические рекомендации со ссылками на статьи нормативных правовых актов.

Цель работы — разработать интеллектуальный программный агент, способный по произвольному словесному описанию сделки определять её гражданско-правовой тип, в диалоговом режиме собирать у пользователя недостающие для подготовки документа сведения и формировать проект договора в формате DOCX вместе с юридическими рекомендациями со ссылками на конкретные статьи нормативных правовых актов. Существующие в области правовых технологий (legal-tech) шаблонные конструкторы не дают правовой оценки описанной сделки с опорой на действующее законодательство, а лишь подставляют пользовательские значения в заранее заготовленный текст из закрытого каталога сценариев. В работе для преодоления этих ограничений применён агентный подход с опорой на проиндексированный корпус российского законодательства.

В рамках работы спроектирован и реализован агент в виде графа состояний, в узлах которого генеративные большие языковые модели (LLM) применяются совместно с методами генерации текста, дополненной поиском по внешней памяти (retrieval-augmented generation, RAG) [1]; внешней памятью служит подготовленный и проиндексированный в векторной базе данных корпус гражданского кодекса РФ и профильных федеральных законов. Программная система реализована на стеке Python с использованием веб-фреймворка FastAPI, фреймворка LangGraph, сервера локального исполнения языковых моделей Ollama, векторной базы данных Qdrant, реляционной СУБД PostgreSQL и объектного хранилища MinIO.

Результатом является целостный программный продукт. Разработанный агент в режиме диалога с пользователем уточняет недостающие сведения о

сделке и формирует проект договора с автоматической самопроверкой его соответствия применимым нормам, сопровождая результат юридическими рекомендациями по заключению описанной сделки. Вокруг агента реализованы веб-интерфейс, платёжный шлюз и панель администратора для управления корпусом нормативных правовых актов и тарифными планами. Эмпирическая оценка поиска по корпусу на специально подготовленном датасете из 54 эталонных описаний сделок показала метрики верхней части выдачи $\text{hit}@3=100\%$ и $\text{precision}@3=91.4\%$ при точности классификации типа сделки 100% ; внешняя экспертная апробация подготовленных артефактов с привлечением трёх экспертов дала средние оценки качества проекта договора и юридических рекомендаций 4.42 и 4.73 по пятибалльной шкале Лайкерта при отсутствии оценок ниже уровня «хорошо» для всех одиннадцати рассмотренных типов сделок. Научная новизна работы — в одновременном сочетании в одной программной системе трёх свойств: работы с произвольной формулировкой сделки от пользователя, опоры на проиндексированный корпус российского законодательства и автоматического контроля качества подготавливаемого проекта договора; в области правовых технологий подобное сочетание ранее не зафиксировано.

Решение ориентировано на физических лиц и субъектов малого предпринимательства, заключающих типовые сделки гражданского оборота. Его применение повышает доступность первичной правовой поддержки, позволяет пользователю самостоятельно проверить, на каких нормах права основаны полученные рекомендации, и снижает риск несоблюдения требований гражданского кодекса РФ к форме и существенным условиям договора.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	5
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	9
1 ПОСТАНОВКА ЗАДАЧИ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНОГО АГЕНТА ДЛЯ ПОДГОТОВКИ ПРОЕКТОВ ДОГОВОРОВ ПО РОССИЙСКОМУ ГРАЖДАНСКОМУ ПРАВУ	12
1.1 Актуальность автоматизированной поддержки граждан и малого бизнеса при заключении сделок гражданского оборота .	12
1.2 Критический анализ существующих программных продуктов в области правовых технологий и подходов к автоматизированной генерации договоров	14
1.3 Цель, задачи и техническое задание на разработку интеллектуального агента генерации проектов договоров	17
2 ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОГО АГЕНТА	21
2.1 Граф состояний агента и его узлы	21
2.2 Поиск по корпусу нормативных правовых актов	26
2.3 Архитектура программной системы и инфраструктура	30
2.4 Экспериментальная оценка и обоснование выбора конфигурации RAG	36
3 РЕЗУЛЬТАТЫ РАЗРАБОТКИ	41
3.1 Условия и сценарии практического применения разработанной системы	41
3.2 Демонстрация работы агента в типовом сценарии	42
3.3 Экспертная оценка качества подготовленных артефактов	45
3.4 Технические характеристики разработанного решения	49
3.5 Выводы по главе	51
ЗАКЛЮЧЕНИЕ	52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	55
ПРИЛОЖЕНИЕ А Состояние агента	58
ПРИЛОЖЕНИЕ Б Полный перечень узлов графа	62
ПРИЛОЖЕНИЕ В Ключевые промпты узлов графа и схема их сборки . .	66
ПРИЛОЖЕНИЕ Г Граф состояний интеллектуального агента	71

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Интеллектуальный программный агент — программная система, которая по входным данным самостоятельно планирует и выполняет последовательность действий для достижения поставленной цели, при необходимости обращаясь к внешним инструментам и пользователю

Граф состояний агента — ориентированный граф, вершины которого соответствуют узлам-обработчикам, а дуги — условным переходам между ними; определяет порядок и условия выполнения шагов обработки запроса

Существенные условия договора — условия, без согласования которых договор по российскому гражданскому праву не считается заключённым: предмет договора, условия, прямо названные таковыми в законе для соответствующего типа сделки, и условия, относительно которых по заявлению одной из сторон должно быть достигнуто соглашение (статья 432 ГК РФ)

Retrieval-Augmented Generation — метод генерации ответа большой языковой моделью, при котором перед генерацией выполняется поиск релевантных фрагментов во внешней памяти, а найденные фрагменты добавляются в контекст модели

Векторная база данных — специализированная система хранения, обеспечивающая эффективный поиск ближайших соседей в пространстве плотных векторных представлений объектов по заданной метрике сходства

Незаключённый договор — договор, в отношении которого не достигнуто соглашение по всем существенным условиям либо не соблюдены требования к форме; не порождает гражданско-правовых обязательств между сторонами

Правовая галлюцинация — ответ большой языковой модели на правовой вопрос, содержащий несуществующие нормы, прецеденты или неверно приписанное содержание реально существующих норм при отсутствии видимых признаков недостоверности

Чанк — смысловой фрагмент исходного документа, получаемый при предварительной обработке корпуса для целей retrieval-augmented generation; типичной единицей чанка служит абзац, раздел или иной осмысленный текстовый блок

Контрольный пункт (checkpoint) — сохранённое состояние графа агента в промежуточной точке его прохождения, позволяющее возобновить обработку после прерывания

Промпт — входной текст для большой языковой модели, задающий контекст, инструкции и предъявляемые к ответу требования

Эмбединг — плотное векторное представление текстового фрагмента фиксированной размерности, получаемое нейросетевой моделью и сохраняющее семантическое сходство исходных текстов в виде близости соответствующих векторов

Бикодер (bi-encoder) — модель векторных представлений, при использовании которой запрос и документ кодируются независимо в векторы фиксированной размерности, а семантическая близость измеряется метрикой между этими векторами; обеспечивает высокую скорость поиска ближайших соседей за счёт предвычисления векторов документов

Кросс-энкодер (cross-encoder) — модель, принимающая на вход пару (запрос, документ) одновременно и возвращающая скаляр-оценку их релевантности; не масштабируется на полнокорпусный поиск, поэтому применяется на этапе переранжирования отобранных бикодером кандидатов

Реранкер (reranker) — модель второго этапа в двухступенчатом поиске, переоценивающая релевантность кандидатов, полученных бикодером, и пересортировывающая их по уточнённой оценке; обычно реализуется как кросс-энкодер

Расширение по графу ссылок — метод обогащения результатов поиска, при котором на этапе предобработки корпуса из текстов фрагментов извлекаются упоминаемые ими ссылки на другие фрагменты, а на этапе поиска по этим ссылкам в результат подгружаются связанные фрагменты с ограничением по числу переходов

LLM-as-a-judge — паттерн использования большой языковой модели в качестве оценщика других ответов или промежуточных артефактов; применяется к задачам, для которых сложно сформулировать формальную метрику

Структурированный вывод — режим работы языковой модели, при котором ответ ограничен переданной JSON Schema; обеспечивает гарантированно валидный машинно-читаемый ответ без необходимости пост-обработки и часто применяется для классификаторов и оценщиков на базе языковых

моделей

Бланкетная отсылка — отсылка одной правовой нормы к другому нормативному акту без указания его конкретной статьи или пункта (например, «в порядке, установленном законом о регистрации»)

Идемпотентность платежа — свойство операции инициирования оплаты, при котором повторный вызов с тем же ключом идемпотентности не создаёт дубликата платежа, а возвращает результат ранее выполненной операции

Узел строгой LLM-фильтрации — консервативный режим LLM-фильтрации релевантности, действующий по принципу «при сомнении — отбросить»; применяется для подавления шума на ранних этапах многошагового поиска

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

DOCX — формат документа Office Open XML

LLM — Large Language Model (большая языковая модель)

RAG — Retrieval-Augmented Generation (генерация текста, дополненная поиском по внешней памяти)

СУБД — система управления базами данных

ГК РФ — Гражданский кодекс Российской Федерации

ВКР — выпускная квалификационная работа

ТЗ — техническое задание

GPU — Graphics Processing Unit (графический процессор)

API — Application Programming Interface (программный интерфейс приложения)

MTEB — Massive Text Embedding Benchmark

ruMTEB — Russian Massive Text Embedding Benchmark

BGE — BAAI General Embedding

НПА — нормативный правовой акт

JSON — JavaScript Object Notation

HTTP — HyperText Transfer Protocol

HTML — HyperText Markup Language

REST — Representational State Transfer

WebSocket — двунаправленный сетевой протокол

OAuth — Open Authorization

ER — Entity-Relationship (сущность-связь)

ACID — Atomicity, Consistency, Isolation, Durability

S3 — Simple Storage Service (объектное хранилище с программным интерфейсом Amazon S3)

PCI DSS — Payment Card Industry Data Security Standard

CSS — Cascading Style Sheets

VRAM — Video Random Access Memory (видеопамять)

ВВЕДЕНИЕ

В современном гражданском обороте Российской Федерации миллионы сделок ежедневно заключаются между физическими и юридическими лицами, причём существенная доля таких сделок относится к типовым массовым правоотношениям — розничной купле-продаже, бытовому подряду, возмездному оказанию услуг, найму жилых помещений, прокату, дарению, займу, сделкам с недвижимым имуществом. Согласно статье 432 Гражданского кодекса Российской Федерации, договор считается заключённым, если между сторонами достигнуто соглашение по всем его существенным условиям в требуемой форме; нарушение этих требований влечёт за собой признание договора незаключённым и сопряжённые с этим финансовые и временные потери для сторон. Между тем большинство участников гражданского оборота не имеет юридического образования, а квалифицированная юридическая помощь при подготовке бытовых договоров экономически малодоступна. Сложившиеся к настоящему моменту шаблонные конструкторы договоров не дают правовой оценки описанной сделки с опорой на действующее законодательство, а лишь подставляют пользовательские значения в заранее заготовленный текст из закрытого каталога сценариев; универсальные большие языковые модели работают со свободной формулировкой запроса, но систематически продуцируют правовые галлюцинации и не привязаны к актуальной редакции законодательства; существующие в правовой сфере системы генерации, дополненной поиском по внешней памяти (retrieval-augmented generation, RAG) [1], в основном ограничиваются сценарием «вопрос-ответ» и не подготавливают полноценный проект договора. Совмещение массовой потребности в первичной правовой поддержке и сложившейся технологической базы — больших языковых моделей (large language models, LLM), методов RAG, открытых векторных баз данных, средств локального исполнения языковых моделей, — определяет актуальность настоящей работы.

Объект исследования — процесс автоматизированной подготовки проектов договоров по российскому гражданскому праву по словесному описанию планируемой сделки. Предмет исследования — архитектура и программная реализация интеллектуального агента, обрабатывающего

произвольную пользовательскую формулировку сделки с опорой на проиндексированный корпус российского законодательства и с автоматическим контролем качества подготавливаемого проекта договора.

Цель работы — разработать интеллектуальный программный агент, способный по произвольному словесному описанию сделки определять тип соответствующего ей гражданского-правового договора, в диалоговом режиме собирать у пользователя недостающие для подготовки документа сведения и формировать проект договора в формате DOCX вместе с юридическими рекомендациями со ссылками на конкретные статьи нормативных правовых актов.

Для достижения поставленной цели в настоящей работе решаются следующие задачи:

- а) сформировать корпус нормативных правовых актов российского договорного права и реализовать его предобработку, разбиение на чанки и индексацию в векторной базе данных;
- б) спроектировать формальную модель агента в виде графа состояний с условными переходами, в узлах которого выполняются отдельные шаги обработки запроса (классификация типа сделки, проверка общих норм, поиск релевантных специальных норм, проверка достаточности данных, генерация проекта договора, его валидация и формирование рекомендаций);
- в) реализовать механизм диалогового уточнения недостающих сведений: при нехватке данных для согласования существенных условий договора агент формулирует пользователю целевой уточняющий вопрос и продолжает обработку после получения ответа;
- г) реализовать автоматическую итеративную проверку сгенерированного проекта договора на полноту существенных условий и непротиворечивость применимым нормам с возможностью повторной генерации при обнаружении замечаний;
- д) разработать модуль генерации файла договора в формате DOCX с надлежащим оформлением документа делового стиля;
- е) разработать веб-интерфейс пользователя и панель администратора для управления корпусом нормативных правовых актов и каталогом поддерживаемых типов сделок;

- ж) обеспечить контейнерное развёртывание программной системы и её работоспособность на локальном оборудовании без передачи пользовательских данных во внешние сервисы;
- и) провести внешнюю экспертную оценку качества подготовленных системой артефактов с привлечением практикующих юристов.

В работе применяются методы анализа предметной области (нормативно-правовой анализ положений Гражданского кодекса Российской Федерации, регламентирующих заключение договора; обзор существующих программных решений в области правовых технологий и научной литературы по применению больших языковых моделей к юридическим задачам); методы проектирования программных систем (формализация поведения агента как ориентированного графа состояний с условными переходами; декомпозиция функций системы на независимые узлы графа); методы разработки систем поиска, дополненной нейросетевыми эмбедингами и реранкерами (RAG-архитектура с двухступенчатой выдачей и LLM-фильтром); методы эмпирической оценки качества (количественные метрики качества поиска на эталонном датасете, экспертная апробация конечных артефактов с использованием пятибалльной шкалы Лайкерта [2]).

Научная новизна работы заключается в построении интеллектуального агента, объединяющего в одной программной системе три свойства: работу с произвольной формулировкой сделки от пользователя, опору на проиндексированный корпус российского законодательства и автоматический контроль качества подготавливаемого проекта договора. Подобное сочетание свойств на момент выполнения настоящей работы в открытой литературе и в практических программных продуктах в области правовых технологий не зафиксировано.

Практическая значимость работы определяется реализацией предложенного подхода в виде целостного программного продукта, применимого для подготовки договоров по бытовым сделкам гражданского оборота физическими лицами и субъектами малого предпринимательства. Внешняя экспертная апробация конечных артефактов с привлечением практикующих юристов подтверждает приемлемый уровень качества проектов договоров и сопровождающих рекомендаций.

1 ПОСТАНОВКА ЗАДАЧИ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНОГО АГЕНТА ДЛЯ ПОДГОТОВКИ ПРОЕКТОВ ДОГОВОРОВ ПО РОССИЙСКОМУ ГРАЖДАНСКОМУ ПРАВУ

1.1 Актуальность автоматизированной поддержки граждан и малого бизнеса при заключении сделок гражданского оборота

В современном гражданском обороте Российской Федерации количество ежедневно заключаемых договоров между физическими и юридическими лицами исчисляется миллионами. К наиболее массовым типам сделок относятся розничная купля-продажа, бытовой подряд, возмездное оказание услуг, наём жилых помещений, прокат, дарение, заём, а также сделки с недвижимым имуществом. Согласно статье 432 Гражданского кодекса Российской Федерации [3; 4] (далее — ГК РФ), договор считается заключённым, если между сторонами в требуемой форме достигнуто соглашение по всем его существенным условиям, перечень которых зависит от типа договора и закреплён в соответствующих главах второй части ГК РФ. Если эти требования нарушены, договор может быть признан незаключённым: стороны теряют возможность принудительно требовать друг от друга исполнения, переданное имущество или денежные средства приходится возвращать, а возникающие из этого споры порождают дополнительные временные и финансовые издержки.

Между тем большая часть участников гражданского оборота не имеет юридического образования. На практике значительная доля бытовых договоров заключается либо устно, либо на основе типовых шаблонов, скачанных из открытых источников. В подготовленных таким образом документах систематически встречаются дефекты: неконкретно определён предмет договора, отсутствует порядок расчётов, не указан срок исполнения, неверно идентифицирован тип сделки, не учтены специальные требования к форме, такие как нотариальное удостоверение или государственная регистрация. Подобные ошибки обычно остаются неочевидными для сторон до возникновения спора и в этот момент уже не могут быть устранены без существенных потерь.

Альтернативой самостоятельной подготовке могло бы быть обращение к практикующему юристу, однако для большинства граждан и индивидуальных

предпринимателей такая возможность экономически малодоступна: стоимость подготовки даже типового договора в специализированной юридической фирме нередко сопоставима со стоимостью самой сделки, а для мелких бытовых сделок может её превосходить. По этой причине повсеместной практикой остаётся самостоятельная подготовка договоров без квалифицированной правовой оценки, что порождает массовый источник правовых рисков для участников оборота. Сложившаяся ситуация делает практически значимой задачу автоматизированной поддержки граждан и субъектов малого предпринимательства при заключении сделок.

Под автоматизированной поддержкой в настоящей работе понимается программная система, которая по словесному описанию планируемой сделки способна без участия юриста выполнять полный цикл подготовки документа: определять применимый гражданско-правовой тип договора, проверять наличие препятствий по общим нормам договорного права, собирать у пользователя недостающие сведения для согласования существенных условий, формировать проект договора в надлежащей форме и сопровождать его рекомендациями с указанием конкретных норм права, ключевых рисков и требований к оформлению. В отличие от шаблонных конструкторов договоров, такая система должна работать с произвольной формулировкой сделки от пользователя и опираться на действующее законодательство, а не на заранее заготовленный ограниченный набор сценариев.

Появление в последние годы больших языковых моделей (large language models, LLM) [5] и методов генерации текста, дополненной поиском по внешней памяти (retrieval-augmented generation, RAG) [1], создало технологические предпосылки для решения такой задачи. Указанные методы позволяют принимать на входе произвольно сформулированный текст пользователя и выдавать ответ, опирающийся на актуальный корпус нормативных правовых актов. Тем самым автоматизированная поддержка перестаёт быть простым «заполнением полей шаблона» и превращается в инструмент, способный обрабатывать сделки, не предусмотренные шаблонным конструктором, и одновременно сопровождать результат проверяемыми ссылками на источники права.

Таким образом, актуальность темы настоящей работы определяется одновременным наличием нескольких факторов. С одной стороны, существует массовая потребность граждан и субъектов малого

предпринимательства в первичной правовой поддержке при заключении сделок гражданского оборота, тогда как квалифицированная юридическая помощь для бытовых сделок небольшой стоимости экономически малодоступна, а самостоятельная подготовка договоров без юридического образования сопряжена с высоким риском правовых дефектов. С другой стороны, к настоящему моменту сложилась доступная технологическая база — большие языковые модели, методы RAG, открытые векторные базы данных и средства локального исполнения языковых моделей, — позволяющая реализовать качественно новый класс инструментов автоматизированной правовой поддержки. Совмещение этих обстоятельств и определяет актуальность настоящей работы.

1.2 Критический анализ существующих программных продуктов в области правовых технологий и подходов к автоматизированной генерации договоров

Сложившиеся к настоящему моменту подходы к автоматизированной подготовке договоров можно условно разделить на три класса, последовательно отражающих развитие технологий обработки естественного языка: программные продукты на основе фиксированных шаблонов, представляющие первое поколение legal-tech; сервисы на основе больших языковых моделей без обращения к внешним источникам права; и более новые гибридные системы, сочетающие большие языковые модели с поиском по внешней памяти. Каждый из этих классов имеет свои сильные стороны и ограничения, которые целесообразно рассмотреть применительно к задаче подготовки договоров по российскому гражданскому праву.

Программные продукты первого класса — шаблонные конструкторы договоров — представлены на российском рынке такими сервисами, как «Конструктор договоров» в составе справочно-правовых систем «КонсультантПлюс» и «Гарант», «Документовед» и «Doczilla». Все они построены на схожем принципе: пользователь выбирает из закрытого перечня тип договора и заполняет фиксированный набор полей формы (стороны, предмет, цена, срок и иные заранее предусмотренные шаблоном параметры), а система подставляет введенные значения в заготовленный текст договора. Достоинство такого подхода — предсказуемость и проверенность результата: каждый шаблон один раз проверен юристом и многократно тиражируется без

потери качества. Однако применительно к задаче, сформулированной в предыдущем подразделе, шаблонные конструкторы имеют принципиальные ограничения. Во-первых, они работают только с заранее предусмотренным набором сценариев: при отклонении ситуации пользователя от типового случая (нестандартный предмет, нестандартный субъектный состав, специальные условия) подставить корректные значения в шаблон невозможно. Во-вторых, пользователь должен сам определить, какой тип договора ему нужен; для непрофессионала эта задача нередко и представляет основную сложность. В-третьих, такие системы дают только сам документ, без сопроводительных рекомендаций о применимых нормах, обязательных формальностях и характерных рисках.

Второй класс решений составляют сервисы на основе универсальных больших языковых моделей — ChatGPT, Claude, Gemini, GigaChat, YandexGPT, — применяемые непосредственно в качестве чат-ботов для генерации договоров. Пользователь даёт модели свободный текстовый запрос, например «составь договор найма однокомнатной квартиры на год», и языковая модель выдаёт ответ. Этот подход снимает главное ограничение шаблонных конструкторов — жёсткость входного формата: модель принимает произвольную формулировку и возвращает связный текст. Однако в правовой области у такого подхода обнаруживаются собственные систематические недостатки. Большие языковые модели общего назначения склонны к так называемым правовым галлюцинациям: модель уверенно ссылается на статьи кодексов, постановления и прецеденты, содержание которых либо не существует, либо приписано по ассоциации. В исследовании Стэнфордского университета на массиве запросов по федеральному праву США доля галлюцинированных ответов составила от 58 % для ChatGPT 4 (сильнейшей на момент исследования модели) до 88 % для Llama 2; при этом модели не отличают собственные достоверные ответы от выдуманных и преподносят и те и другие с одинаковой уверенностью [6]. Для систематической оценки качества языковых моделей на юридических задачах создан бенчмарк LegalBench, объединяющий более 160 задач по шести видам правового рассуждения и фиксирующий, что даже сильнейшие модели общего назначения существенно отстают от человека на сложных правовых сценариях [7]. Помимо высокой доли фактических ошибок, ответ универсальной языковой модели не привязан к актуальной редакции

законодательства и не содержит проверяемой ссылочной базы. Отдельный класс ограничений связан с режимом обработки данных у поставщика подобной публичной LLM: пользовательский запрос на подготовку договора по умолчанию раскрывает поставщику персональные сведения и имущественные намерения сторон, а пользовательские соглашения большинства таких сервисов в стандартной конфигурации допускают использование передаваемых данных для обучения моделей и иных производных целей, что для правовой задачи плохо совместимо с разумными ожиданиями пользователя относительно конфиденциальности.

Третий класс — сервисы, в которых большие языковые модели дополнены поиском по внешней памяти (retrieval-augmented generation, RAG) [1; 8]. На входе пользовательский запрос преобразуется в векторное представление, по нему производится поиск в индексе нормативных правовых актов, а найденные фрагменты добавляются в контекст языковой модели. Это существенно снижает уровень галлюцинаций: модель отвечает по найденным выдержкам из законодательства, а не по своей внутренней «памяти». В научной литературе подобный подход применительно к договорному праву реализован, в частности, в работе [9] для марокканского Кодекса обязательств и контрактов: авторы показали, что RAG-чатбот достигает близкой к единице точности по метрике соответствия ответа найденному контексту и обходит универсальные языковые модели на тех же задачах вопрос-ответ. Вместе с тем существующие RAG-системы в legal-tech, как правило, ограничиваются сценарием «вопрос-ответ»: договор как структурированный артефакт с обязательным составом разделов такие системы не формируют, а автоматическая проверка качества сгенерированного содержания в них не реализуется. В настоящей работе предлагается дополнить RAG-подход именно этими механизмами — генерацией полноценного проекта договора и его автоматической итеративной проверкой.

Отдельно следует упомянуть растущий класс агентных систем, в которых большая языковая модель оркестрируется внешним фреймворком и выполняет многошаговое рассуждение с вызовами инструментов: поиска, валидации, обращения к пользователю за уточнением. Современные обзоры таких систем [10] классифицируют их по способу планирования действий, организации памяти и доступу к внешним инструментам и фиксируют, что агентная архитектура существенно повышает надёжность языковой модели в

сложных многошаговых задачах по сравнению с одноразовым запросом. Однако применение агентного подхода к задаче подготовки проектов договоров по российскому гражданскому праву в открытой литературе не зафиксировано, а в практических legal-tech-продуктах распространения пока не получило.

Сопоставление перечисленных подходов выявляет научно-практический пробел. Шаблонные конструкторы обеспечивают надёжный результат, но не работают с произвольным описанием сделки и не дают её правовой оценки. Универсальные языковые модели понимают свободный текст, но галлюцинируют нормы и не привязаны к актуальному законодательству. Существующие RAG-системы в правовой сфере снимают наиболее массовые формы галлюцинаций, но в основном ограничиваются вопрос-ответными сценариями и не реализуют ни самостоятельной подготовки проекта договора как структурированного документа, ни автоматической проверки полноты существенных условий, ни диалогового сбора недостающих данных у пользователя. Открытого решения, которое одновременно принимало бы произвольное словесное описание сделки на русском языке, опиралось на проиндексированный корпус российских нормативных правовых актов, уточняло у пользователя недостающие сведения в диалоговом режиме, формировало проект договора надлежащей структуры и автоматически проверяло его на полноту существенных условий и непротиворечивость применимым нормам, на момент выполнения настоящей работы не существует. Восполнение этого пробела составляет содержание настоящей выпускной квалификационной работы.

1.3 Цель, задачи и техническое задание на разработку интеллектуального агента генерации проектов договоров

Целью настоящей выпускной квалификационной работы является разработка интеллектуального программного агента, способного по произвольному словесному описанию сделки определять её гражданско-правовой тип, в диалоговом режиме собирать у пользователя недостающие для подготовки документа сведения и формировать проект договора в формате DOCX вместе с юридическими рекомендациями со ссылками на конкретные статьи нормативных правовых актов. Принципиальное отличие разрабатываемого решения от рассмотренных в

подразделе 1.2 классов систем заключается в сочетании трёх свойств в одном агенте: работа с произвольной формулировкой сделки от пользователя, опора на проиндексированный корпус российского законодательства и автоматическая итеративная проверка сгенерированного проекта договора на полноту существенных условий и непротиворечивость применимым нормам.

Для достижения поставленной цели в настоящей работе решаются следующие задачи:

- а) сформировать корпус нормативных правовых актов, включающий часть первую и часть вторую ГК РФ и профильные федеральные законы, и реализовать его предобработку, разбиение на чанки и индексацию в векторной базе данных;
- б) спроектировать формальную модель агента в виде графа состояний с условными переходами, в узлах которого выполняются отдельные шаги обработки запроса: классификация типа сделки, проверка общих норм, поиск релевантных специальных норм, проверка достаточности данных, генерация проекта договора, его валидация и формирование рекомендаций;
- в) реализовать механизм диалогового уточнения недостающих сведений: при нехватке данных для согласования существенных условий договора агент должен формулировать пользователю целевой уточняющий вопрос и продолжать обработку после получения ответа;
- г) реализовать автоматическую итеративную проверку сгенерированного проекта договора на полноту существенных условий и непротиворечивость применимым нормам с автоматической повторной генерацией при обнаружении замечаний;
- д) разработать модуль генерации файла договора в формате DOCX с надлежащим оформлением документа;
- е) разработать веб-интерфейс пользователя и панель администратора для управления корпусом нормативных правовых актов;
- ж) обеспечить контейнерное развёртывание программной системы и её работоспособность на локальном оборудовании без передачи пользовательских данных во внешние сервисы;
- и) провести экспертную оценку качества работы агента с привлечением практикующих юристов: оценить юридическую

корректность сгенерированных проектов договоров и сопровождающих рекомендаций; обоснование выбора RAG-архитектуры подкрепить опубликованными результатами исследований, фиксирующими деградацию качества языковых моделей на правовых задачах без обращения к внешней нормативной памяти.

Техническое задание на разработку определяет следующие функциональные требования. Система должна принимать на входе произвольный словесный запрос пользователя на русском языке, описывающий планируемую сделку, и поддерживать основные типы договоров, поименованные в части второй ГК РФ (купля-продажа, аренда, наём жилого помещения, подряд, возмездное оказание услуг, дарение, заём, перевозка и иные распространённые типы сделок гражданского оборота). По каждому обработанному запросу система обязана выдать два артефакта: проект договора в формате DOCX, соответствующий типовой структуре делового документа с печатными полями для рукописного заполнения, и сопровождающие юридические рекомендации в формате Markdown. Рекомендации должны содержать сведения о требуемой форме сделки, существенных условиях, особенностях оформления (включая необходимость нотариального удостоверения или государственной регистрации, если они предусмотрены законом) и ключевых рисках; каждое утверждение должно сопровождаться ссылкой на конкретную статью нормативного правового акта в формате «Источник, статья N». При нехватке сведений для согласования существенных условий система должна задавать пользователю целевой уточняющий вопрос и продолжать обработку после получения ответа. После генерации проекта документа должна выполняться автоматическая проверка его содержания с возможностью повторной генерации при обнаружении замечаний.

Нефункциональные требования к системе включают контейнерное развёртывание инфраструктурных компонентов средствами Docker Compose, обеспечивающее воспроизводимость окружения; поддержку двух режимов работы языковой модели — локального исполнения через сервер Ollama при наличии GPU достаточного объёма (что позволит обрабатывать и хранить пользовательские данные без передачи во внешние сервисы) либо подключения к внешнему OpenAI-совместимому API при ограниченных

локальных ресурсах, — задаваемых конфигурацией без изменения исходного кода; среднее время отклика на полный цикл подготовки документа в пределах единиц минут; многопользовательский режим работы с аутентификацией через внешних провайдеров идентификации (Google, Яндекс) и хранением истории обращений каждого пользователя в реляционной базе данных; наличие административной панели, позволяющей расширять корпус нормативных правовых актов и управлять тарифными планами без изменений в исходном коде системы.

2 ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОГО АГЕНТА

2.1 Граф состояний агента и его узлы

В основе разработанного решения лежит представление агента как конечного ориентированного графа состояний с условными переходами. Формально агент задаётся четвёркой

$$\mathcal{A} = (\Sigma, V, \delta, \sigma_0), \quad (1)$$

где Σ — множество допустимых значений состояния обработки запроса, V — конечное множество узлов-обработчиков, $\delta : \Sigma \times V \rightarrow V$ — функция перехода, $\sigma_0 \in \Sigma$ — начальное состояние, формируемое из входного сообщения пользователя и инжектируемого контекста. Каждый узел $v \in V$ определяет отображение

$$f_v : \Sigma \rightarrow \Sigma, \quad (2)$$

которое принимает текущее состояние, выполняет свою операцию (один или несколько вызовов LLM, обращение к векторной базе данных, формирование служебных полей) и возвращает обновлённое состояние. Реализация графа выполнена на фреймворке LangGraph [11], поддерживающем декларативную сборку графа с возможностью прерывания обработки на запросы уточнений у пользователя и последующего возобновления с восстановлением промежуточного состояния из контрольного пункта (checkpoint). Все промежуточные вызовы LLM для классификации, проверок, фильтрации и валидации выполняются в режиме структурированного вывода: модель принимает на вход JSON-схему ожидаемого ответа и возвращает соответствующий ей объект без дополнительной пост-обработки. В текущей конфигурации все вызовы LLM выполняются моделью `gemma4:31b-cloud`, доступной через сервер локального исполнения Ollama. Состояние агента описывается типизированной структурой — словарём с предопределённым набором именованных полей; полная спецификация полей с указанием типа и назначения приведена в приложении А.

Граф $G = (V, E)$ содержит двадцать один узел, объединённый в семь функциональных фаз: маршрутизация входящего сообщения; классификация сделки; ранние проверки правовой допустимости; поиск по корпусу;

диалоговое уточнение и проверки достаточности данных; генерация проекта договора и его итеративная валидация; формирование рекомендаций, обогащение уточняющими ссылками и сохранение итогового результата. Полная схема графа со всеми узлами, переходами и точками диалогового взаимодействия с пользователем приведена в приложении Г на рисунке Г.1; там же даны условные обозначения формы узлов, цвета подграфов и типов стрелок. Развёрнутая таблица узлов с указанием входов, выходов и использованных промптов вынесена в приложение Б, тексты ключевых промптов и схема их сборки — в приложение В.

Маршрутизация входящего сообщения и проверка тарифа. Узел `route_user_message` получает входящее сообщение и принимает решение, в какую ветвь графа отправить запрос: новая сделка с полным циклом обработки (`regenerate`); вопрос или комментарий по уже сгенерированному результату (`followup`); точечная правка существующего договора (`edit`). Если намерение определено как редактирование, но контекст пользователя не позволяет редактировать готовые договоры (бесплатный тариф), управление передаётся узлу `gate_free_plan`, который формирует сообщение о необходимости перейти на платный тариф и завершает текущий проход через граф, — тем самым биллинговое ограничение реализуется на уровне узла графа, а не HTTP-эндпоинта.

Классификация типа сделки. Узел `classify_deal` реализован как двухступенчатая стратегия. Сначала вызывается основной классификатор: модель в режиме структурированного вывода возвращает поля `deal_type` (одно из закрытого перечня поддерживаемых типов), `confidence` и при необходимости `clarification_question`. При низкой уверенности активируется субагент классификации с RAG-поддержкой: внутри изолированной подзадачи с ограниченным бюджетом запросов к коллекции `npa_primal` он на каждой итерации формулирует поисковый запрос, получает короткие резюме найденных норм и уточняет классификацию по фактам из корпуса. Принципиальное отличие субагента от обычного классификатора — он видит факты из корпуса, а не только опору на априорное знание модели, и поэтому существенно реже путает типы с похожими названиями (например, бытовой подряд и возмездное оказание услуг). По итогам работы классификатора возможны три исхода: тип определён уверенно — обработка продолжается с узла проверки общих норм;

описанная сделка не относится ни к одному из поддерживаемых типов — активируется узел `inform_unsupported_deal`, формирующий вежливое сообщение об отсутствии поддержки и завершающий обработку; уверенности по-прежнему недостаточно — активируется ветвь диалогового уточнения у пользователя.

Ранние проверки правовой допустимости. Узел `load_general_norms` запускается до классификации и однократно подгружает все чанки коллекции `nra_general` в состояние — общие положения договорного права применимы к любому типу сделки, и эффективнее держать их в состоянии целиком, а не запускать векторный поиск на каждый запрос. Узел `check_general_norms` оценивает наличие прямых препятствий по общим нормам (запрет сделки, нарушение обязательной формы, недопустимый предмет, отсутствие обязательного согласия) и при их обнаружении направляет граф в узел `inform_user`, который формирует пояснительное сообщение пользователю; обработка на этом завершается. При отсутствии препятствий граф переходит к поиску специальных норм (узел `retrieve_norms`, реализация которого подробно рассмотрена в подразделе 2.2).

Проверки формы и достаточности данных. Узел `check_written_form` определяет необходимость письменной формы на основе уже извлечённых норм. В сочетании с пользовательской политикой подготовки документа возможны три исхода: при политике «всегда генерировать» — обработка переходит к проверке достаточности данных; при политике «только если требуется законом» — если письменная форма не требуется, обработка переходит сразу к формированию рекомендаций без генерации документа; при политике по умолчанию «спрашивать» — если закон допускает устную форму, инициируется диалоговое уточнение у пользователя о необходимости подготовки печатного проекта договора. Узел `check_data_sufficiency` оценивает достаточность данных в описании сделки и при их нехватке формирует один связный уточняющий вопрос; промпт узла учитывает пользовательскую настройку об обработке персональных данных: если пользователь отключил их сбор, узел исключает ФИО, паспорт, ИНН и адреса из списка обязательных полей, заменяя их в готовом документе печатными полями для рукописного заполнения. Дополнительный узел `optional_contract_answer` служит точкой

возобновления обработки после получения ответа пользователя «да/нет» на вопрос о подготовке документа: при положительном ответе обработка продолжается с проверки достаточности данных, при отрицательном — сразу переходит к формированию рекомендаций.

Генерация проекта договора и итеративная валидация. Узел `generate_contract` принимает на вход тип сделки, найденные нормы, структуру сделки и стандартный HTML-шаблон делового договора (типовые девять разделов: предмет, срок, права и обязанности, цена и расчёты, ответственность, изменение и расторжение, разрешение споров, заключительные положения, реквизиты и подписи). Промпт явно перечисляет разрешённые HTML-теги и запрещает Markdown и пояснения вне разметки; важное правило — в самом тексте договора не упоминать статьи закона: договор должен формулироваться обычным языком, а соответствующие нормы и риски выносятся в раздел рекомендаций. Для недостающих сведений промпт предписывает использовать печатные поля (последовательности подчёркиваний), что позволяет распечатать договор и заполнить вручную при подписании. Узел `validate_contract` проверяет существенные условия и общую правовую корректность проекта, возвращая список конкретных замечаний с указанием раздела договора. При обнаружении замечаний и не исчерпанном лимите итераций граф через узел `handle_validation_error` возвращается на проверку достаточности данных и далее к повторной генерации, причём список замечаний инжектируется в состояние и попадает в контекстный промпт следующей попытки. На число итераций повторной генерации наложено ограничение $iter \leq iter_{\max}$ (в текущей конфигурации $iter_{\max} = 10$); при исчерпании лимита граф переходит к формированию рекомендаций с пометкой о невалидном проекте — такая мягкая деградация предпочтительнее отказа, поскольку рекомендации сохраняют ценность и при невалидном договоре.

Рекомендации, обогащение и сохранение. Узел `generate_recommendations` формирует юридический отчёт в формате Markdown; промпт предписывает структуру (обязательность письменной формы, существенные условия, особое оформление, основные риски) и требует, чтобы каждое утверждение сопровождалось ссылкой на источник в формате [Источник, статья]. Следующий узел `enrich_recommendations` устраняет бланкетные отсылки на

нормативные акты без указания конкретной статьи через дополнительные точечные обращения в коллекцию `nra_secondary`. Узел `save_result` преобразует HTML-проект в файл DOCX (стек `html2docx+python-docx`: первая библиотека преобразует разрешённое подмножество HTML в объектную модель OpenXML, вторая применяет оформление — шрифт Times New Roman 12 пт, полуторный межстрочный интервал, поля по 25 мм), сохраняет файл в объектное хранилище MinIO [12] и обновляет статус кейса.

Вспомогательные ветви. Помимо основного пути в графе предусмотрены две вспомогательные ветви. Узел `edit_contract` обслуживает точечную правку готового документа: получает на вход текущий HTML-проект и запрос пользователя на изменение, возвращает обновлённый HTML и список затронутых разделов. После редактирования управление передаётся узлу `validate_contract` в особом режиме, который не запускает цикл регенерации даже при обнаружении замечаний, а сразу направляет результат в узел сохранения. Ветвь обработки дополнительных вопросов представлена парой узлов: `followup_response` принимает решение о возможности немедленного ответа на основе уже найденных норм и сгенерированных материалов; при необходимости обращения к корпусу за новыми нормами управление передаётся узлу `followup_subagent_search`, который реализует изолированный итеративный поиск по коллекции `nra_secondary` с ограничением сверху на число попыток.

Механизм диалогового уточнения. Особую роль играют переходы, прерывающие обработку на запросы уточнений у пользователя. Все три источника уточнений — неопределённость классификации (узел `classify_deal`), необходимость подтверждения о подготовке печатного документа (узел `check_written_form`) и нехватка существенных сведений (узел `check_data_sufficiency`) — передают управление общему узлу `ask_clarification`, который выводит сформированный вопрос в чат и завершает текущий проход через граф. В состоянии при этом сохраняется поле `clarification_stage`, фиксирующее, какой именно узел инициировал уточнение. При поступлении следующего сообщения пользователя средствами LangGraph восстанавливается ранее сохранённое состояние, в него добавляется текст ответа, и обработка возобновляется в узле, соответствующем зафиксированной стадии: `classify_deal` для

уточнения классификации, `optional_contract_answer` для ответа «да/нет» о подготовке документа, `check_data_sufficiency` для пополнения существенных сведений. Тем самым исключается повторное выполнение ранее пройденных шагов классификации, поиска норм и ранних проверок, в типичном сценарии стоящих на порядок дороже самого вопроса уточнения.

2.2 Поиск по корпусу нормативных правовых актов

Поиск отвечает в графе агента за извлечение релевантных норм права из проиндексированного корпуса и за их передачу в узлы проверки и генерации в виде упорядоченного набора фрагментов уровня отдельной статьи или её подпункта вместе со структурированными метаданными источника. От его качества напрямую зависят точность правовой оценки сделки и обоснованность готового договора. Поиск построен на пяти проектных решениях: разнесение корпуса по нескольким коллекциям, выбор модели эмбедингов с инструкционной обёрткой запроса, двухступенчатый отбор с ранкером, расширение результата по графу взаимных ссылок чанков и финальная LLM-фильтрация выдачи.

Корпус нормативных правовых актов разнесён по трём независимым коллекциям в векторной базе данных Qdrant. Назначение коллекций: `nra_general` содержит общие положения договорного права (понятие сделки, форму, существенные условия, общие основания заключения и прекращения договора); `nra_primal` — специальные нормы по конкретным типам сделок; `nra_secondary` — акты, на которые ссылаются специальные нормы из `nra_primal`. Состав каждой из коллекций определяется администратором через панель управления корпусом и может меняться без пересборки приложения. В тестовой конфигурации, использованной при экспериментальной оценке системы (подраздел 2.4), в `nra_general` проиндексирована часть первая ГК РФ [3]; в `nra_primal` — часть вторая ГК РФ [4]; в `nra_secondary` — часть первая и часть вторая ГК РФ, Жилищный кодекс РФ, Федеральный закон «О защите прав потребителей», Федеральный закон «О государственной регистрации недвижимости» и Федеральный закон «О потребительском кредите (займе)». Документы корпуса предварительно разбиваются на смысловые блоки — чанки — на уровне отдельной статьи или статьи с подпунктами; крупные статьи

дополнительно разрезаются по морфологическим границам подпунктов. К каждому чанку прикреплены структурированные метаданные (имя источника, глава, номер статьи, индекс чанка внутри длинной статьи) и массив *references* — список упоминаемых в тексте чанка ссылок на другие нормативные акты, заполняемый отдельным LLM-вызовом на этапе предобработки и используемый далее для расширения результата поиска по графу ссылок.

В качестве модели векторных представлений выбрана *ai-sage/Giga-Embeddings-instruct* [13] — русскоязычная модель эмбедингов размерности 2048, специально обученная для задач поиска и оценки семантической близости текстов. Согласно опубликованным результатам бенчмарка *ruMTEB* [14] — стандартного набора задач для оценки качества русскоязычных моделей эмбедингов, — эта модель показывает наилучшую среднюю оценку среди открытых моделей на задачах информационного поиска для русского языка. Модель относится к классу инструкционно-обучённых: запрос дополнительно оборачивается короткой инструкцией о решаемой задаче в формате

Instruct: *⟨описание задачи⟩*\n Query: *⟨текст запроса⟩*,

без которой точность поиска существенно падает. В разработанной системе строка инструкции задаётся параметром `EMBEDDING_QUERY_INSTRUCTION` и для основной задачи имеет вид «Дан тип и описание сделки, необходимо найти текст, где описывается похожая сделка»; для документов корпуса на этапе индексации обёртка не применяется. Архитектура подсистемы не привязана к конкретной модели эмбедингов жёстко: смена модели выполняется конфигурационным параметром без изменения исходного кода, что было использовано при экспериментальной оценке (подраздел 2.4).

Поиск решает задачу отбора в коллекции из N чанков короткого подмножества, наиболее релевантного запросу. Прямой перебор корпуса с оценкой релевантности каждого чанка средствами LLM при N порядка десятков тысяч вычислительно невозможен, поэтому реализована стандартная для современных систем информационного поиска [15] и, в частности, RAG-систем [16] двухступенчатая процедура: первый этап быстро отбирает широкое множество кандидатов с гарантированным охватом, второй — точно

ранжирует уже отобранный короткий список. На первом этапе применяется бикодер — нейросетевая модель [17], отображающая текст в вектор фиксированной размерности; запрос и чанк кодируются независимо, благодаря чему векторы всех чанков корпуса вычисляются один раз на этапе индексации и сохраняются в векторной базе. Семантическая близость пары измеряется косинусом угла между векторами

$$\text{sim}(q, c_i) = \cos(\mathbf{q}, \mathbf{v}_i) = \frac{\langle \mathbf{q}, \mathbf{v}_i \rangle}{\|\mathbf{q}\| \|\mathbf{v}_i\|}. \quad (3)$$

Для приближённого поиска ближайших соседей по (3) в Qdrant используется индекс на основе иерархических наведируемых малых миров (Hierarchical Navigable Small World, HNSW) [18], что даёт логарифмическую ожидаемую сложность поиска при качестве выдачи, близком к точному поиску полным перебором. Результат первого этапа — 80 чанков с наибольшим значением близости.

На втором этапе работает кросс-энкодерный реранкер. Он принимает пару (запрос, чанк) совместно и возвращает скалярную оценку релевантности, опираясь на полноценное взаимное внимание токенов запроса и чанка внутри одной модели. Это даёт более точное ранжирование, чем бикодер, но не допускает предвычисления: для каждой пары требуется отдельный прогон модели; двухступенчатая схема снимает это противоречие, применяя точную, но медленную модель только к небольшому множеству кандидатов, уже отобранных бикодером. Тем самым первый этап обеспечивает охват по корпусу (recall), а второй — точность ранжирования верхней части выдачи (precision). В разработанной системе поддерживаются два реранкера — BAAI/bge-reranker-v2-m3 [19] и Qwen/Qwen3-Reranker-0.6B [20] — с возможностью переключения конфигурационным параметром; сравнение конфигураций с разными реранкерами выполнено эмпирически в подразделе 2.4.

Помимо чисто семантической близости, релевантность нормы определяется и её положением в сети взаимных ссылок нормативных актов. Статья ГК РФ, описывающая конкретный тип договора, как правило, содержит ссылки на другие статьи того же кодекса или на иные нормативные акты; игнорирование этой структуры приводит к тому, что в результат поиска попадает основная статья по теме, но не попадают её необходимые в правоприменении уточнения. Для устранения этой потери в подсистему

встроено расширение результата по графу ссылок: для каждого из уже отобранных и переранжированных чанков по полю `references` извлекается пара (источник, статья), по этой паре выполняется точечный запрос в коллекцию `npa_secondary` по точному совпадению метаданных (без вычисления эмбединга), и полученные новые чанки используются как входные на следующей ступени. Число чанков, добавляемых на одной ступени, ограничено сверху значением 20, число ступеней — тремя. Логика расширения проиллюстрирована на рисунке 2.1.

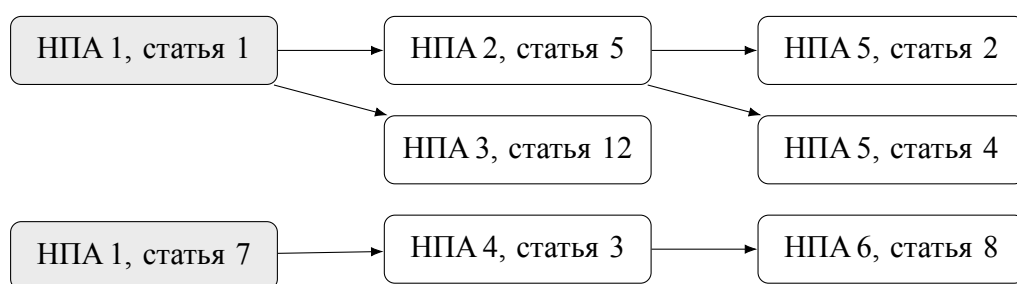


Рисунок 2.1 – Расширение результата поиска по графу взаимных ссылок чанков: показаны исходные чанки (слева, выделены заливкой) и две ступени расширения с ветвлением по ссылкам; всего применяется не более трёх ступеней с лимитом до 20 чанков на каждой

После расширения объединённое множество «первичные чанки $\cup$ чанки расширения» подвергается совместному переранжированию тем же ранкером, что позволяет высокорелевантной цитированной статье обойти слабый хвост первичной выдачи и попасть в верх итогового результата. Финальный размер выдачи ограничен 30 чанками.

Завершает подсистему LLM-фильтр — самостоятельный узел графа, который вызывает LLM в режиме структурированного вывода и просит её вернуть массив идентификаторов тех чанков, которые относятся к описанной сделке. Промпт сформулирован так, что отбрасывать чанк следует только при уверенной нерелевантности; при любом сомнении чанк сохраняется. Такой режим использования модели как оценщика релевантности — частный случай парадигмы LLM-as-a-judge [21]. Чтобы фильтр работал и при больших входных множествах, чанки разбиваются на батчи размера 10, фильтр вызывается на каждом батче независимо, и итоговое множество одобренных чанков получается объединением одобренных в каждом батче. Эмпирическая оценка результатов подсистемы и обоснование текущей конфигурации

приведены в подразделе 2.4.

2.3 Архитектура программной системы и инфраструктура

Программная система спроектирована как набор слабо связанных сервисов, развёртываемых в виде Docker-контейнеров и взаимодействующих по фиксированным сетевым протоколам внутри единой виртуальной сети. Принятые проектные решения по разделению данных по специализированным хранилищам (реляционная СУБД, векторный индекс, объектное хранилище) и асинхронной обработке долгих операций согласуются с типовыми подходами проектирования распределённых систем [22]. Оркестровка ведётся средствами Docker Compose [23]: семь сервисов описаны единым декларативным файлом и поднимаются одной командой; применение Kubernetes для бакалаврской работы признано избыточным, поскольку контейнеризация на уровне Compose обеспечивает воспроизводимость окружения и совместимость с типовыми разработческими и серверными машинами без накладных расходов на освоение кластерной инфраструктуры. На верхнем уровне архитектуры выделяются три категории внутренних компонентов: инфраструктурные сервисы (Ollama для исполнения LLM, Qdrant для векторного поиска, PostgreSQL для реляционных данных, MinIO для объектного хранилища); прикладные сервисы (контейнер LangGraph с компилированным графом и REST-фасадом, контейнер FastAPI с бизнес-логикой и WebSocket-каналом); клиентские компоненты (фронтенд, отдаваемый отдельным nginx-контейнером и исполняемый в браузере пользователя). Помимо внутренних компонентов система взаимодействует с двумя внешними службами, не входящими в периметр развёртывания: провайдерами идентификации Google и Яндекс (вход пользователей по протоколу OAuth 2.0) и платёжным шлюзом ЮKassa (приём платежей по платным тарифным планам и обработка асинхронных уведомлений о статусе платежа). Передача в эти службы ограничена данными, необходимыми для выполнения соответствующей операции, и не включает содержание пользовательского запроса о сделке или сгенерированных артефактов. Между внутренними сервисами установлены прямые сетевые связи внутри Docker-сети, внешний доступ осуществляется через два публикуемых порта — HTTP-API основного приложения и порт фронтенд-сервера, а обращения к внешним службам проходят по публичной сети. Компонентная

архитектура, взаимодействия между сервисами и связи с внешними службами приведены на рисунке 2.2 (внешние службы изображены пунктирной границей).

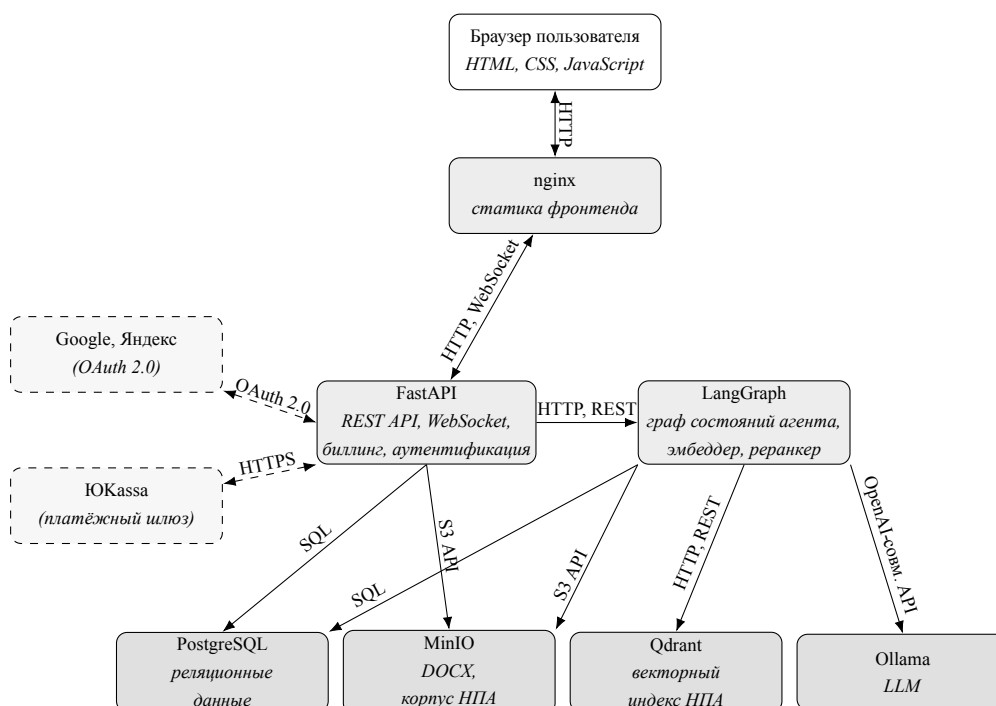


Рисунок 2.2 – Компонентная архитектура программной системы и её внешних зависимостей

Прикладная логика разделена на два процесса. Контейнер `app` запускает основное приложение FastAPI [24] с REST-API, WebSocket-каналом и бизнес-логикой биллинга и аутентификации. Контейнер `langgraph_dev` запускает CLI-сервер `langgraph dev`, который предоставляет REST-API над скомпилированным графом; контейнер собирается из того же образа, что и `app`, и отличается от него только стартовой командой. FastAPI обращается к серверу графа по HTTP: эндпоинт `POST /api/chat` немедленно возвращает `202 Accepted` с идентификаторами созданного сообщения-заполнителя, длительная обработка делегируется фоновой задаче под асинхронной блокировкой, индивидуальной для каждого `case_id`, а сама задача периодически опрашивает REST-фасад LangGraph до завершения прогона. Идентификатор кейса одновременно служит идентификатором потока (`thread`) в LangGraph, что позволяет средствами фреймворка возобновлять обработку после прерывания на запрос уточнения без повторного исполнения уже пройденных узлов. На максимальную продолжительность одного прогона наложено ограничение в 600 с; по его истечении прогон отменяется, статус

сообщения переключается на ошибочный, и кейс помечается как завершившийся с ошибкой. По завершении прогона пользователь получает уведомление по WebSocket-каналу `/api/ws`.

Реляционная схема насчитывает десять связанных таблиц, описывающих основные сущности предметной области; объектно-реляционное отображение выполнено средствами SQLAlchemy. ER-диаграмма базы данных приведена на рисунке 2.3.

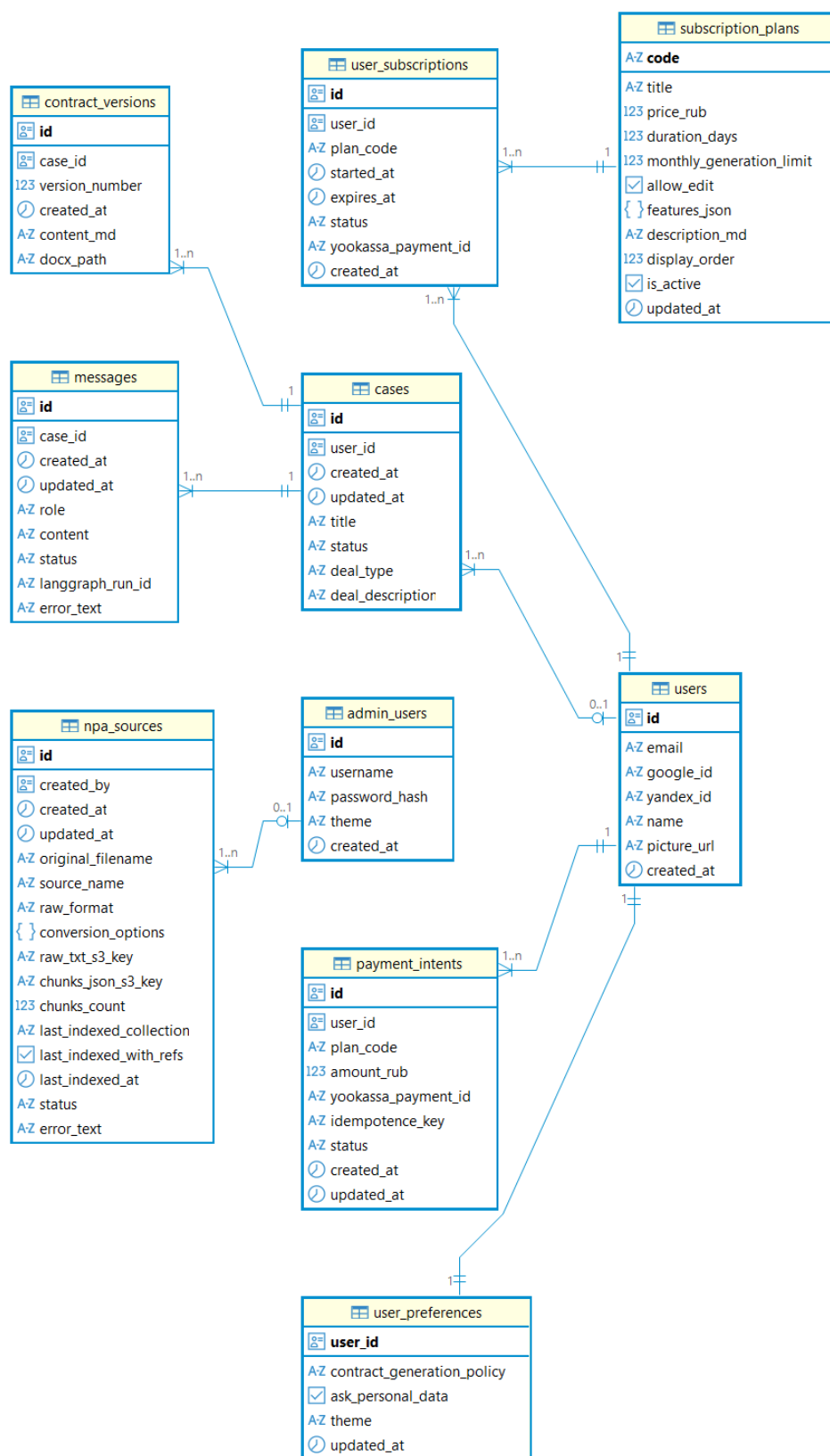


Рисунок 2.3 – ER-диаграмма базы данных программной системы

Выбор технологического стека определяется требованиями технического задания (подраздел 1.3) и характером формальной модели

агента. LangGraph выбран как фреймворк, наиболее соответствующий предложенной модели: линейные цепочки библиотеки LangChain не поддерживают условные переходы, мульти-агентные фреймворки типа CrewAI и AutoGen предназначены для сценариев свободного взаимодействия нескольких ролей, ручная реализация графа на чистом Python потребовала бы повторной реализации механизма прерывания и возобновления. Векторная база данных Qdrant выбрана за поддержку фильтрации по структурированным метаданным (используется при точечной подгрузке актов по графу ссылок), хранение нескольких независимых коллекций (используется для разнесения корпуса) и наличие официального Python-клиента с асинхронной поддержкой. Веб-уровень построен на FastAPI: типизированные схемы через Pydantic, нативный asyncio, готовый WebSocket-стек и автогенерация OpenAPI-спецификации. Реляционная СУБД — PostgreSQL 16: гарантии ACID, нужные биллинговой подсистеме, и поле JSONB для гибких полуструктурированных данных. Объектное хранилище — MinIO [12]: открытая S3-совместимая реализация с подписанными ссылками.

Принципиальное решение о выборе языковой модели — использование открытых моделей с возможностью локального исполнения, а не публичных API крупных коммерческих LLM. Это решение обусловлено характером обрабатываемых данных: пользователь передаёт системе персональные сведения (паспортные данные, ИНН, ОГРН, адреса) и описание планируемой сделки, в ряде случаев финансово существенной. Отправка таких данных в публичный API стороннего поставщика означала бы их доступность этому поставщику в режимах, регулируемых его пользовательским соглашением: для большинства коммерческих LLM-сервисов условия по умолчанию допускают использование пользовательских запросов для обучения и улучшения моделей и иных производных целей. Для правовой задачи, в которой содержание запроса само по себе раскрывает имущественные намерения и личные данные сторон, такой режим обработки несовместим с разумными ожиданиями пользователя относительно конфиденциальности. Использование открытой модели снимает эту проблему: модель и веса доступны для локального запуска внутри периметра эксплуатирующей организации без передачи запросов третьим лицам. В текущем развёртывании в параметре LLM_MODEL указана модель gemma4:31b-cloud — модель размера 31 миллиард параметров в режиме облачной маршрутизации Ollama,

использованная на этапе экспериментальной проверки; архитектура системы при этом не зависит жёстко от Ollama: благодаря OpenAI-совместимости интерфейса возможно подключение и любого другого совместимого провайдера, включая полностью локальный.

Аутентификация пользователей построена на протоколе OAuth 2.0 [25] через библиотеку `authlib` для интеграции с Google и собственную реализацию обмена кода авторизации на токен для Яндекс ID. Аутентификация администраторов реализована отдельным контуром с парами имя пользователя и пароль и хешами `bcrypt`; двухконтурная сессия (одна и та же cookie `Starlette` хранит независимо `user_id` и `admin_id`) позволяет одному физическому пользователю быть одновременно и обычным пользователем, и администратором без двойного входа. Биллинговая подсистема представляет собой замкнутый домен с собственными бизнес-правилами: тарифные планы хранятся в таблице `subscription_plans` и редактируются администратором; зарезервированный код `free` соответствует бесплатному плану с количественным лимитом на генерации и запретом на редактирование, который реализуется на двух уровнях — инъекция признака `allow_edit=false` в состояние агента и узел программной преграды внутри графа. Платёжный поток реализован через ЮKassa с уникальным ключом идемпотентности на каждое намерение оплаты; выбор именно этого провайдера определяется его PCI DSS-совместимостью.

Административный контур позволяет управлять корпусом нормативных правовых актов: загруженный администратором текст нормативного акта сохраняется в бакет `admin-npa` MinIO, далее запускается деление на чанки (морфологическое либо с LLM-извлечением ссылок в массив `references`), и отдельной задачей администратор инициирует индексацию обработанного акта в одну из коллекций Qdrant. Кроме работы с корпусом административный контур позволяет редактировать каталог поддерживаемых типов сделок, управлять тарифными планами и просматривать список учётных записей. Веб-интерфейс пользователя реализован на ванильном JavaScript без сборочного шага: разметка и стили статически отдаются `nginx`-контейнером, маршрутизация между видами выполняется переключением CSS-видимости, состояние UI собрано в единый объект и обновляется в реакции на WebSocket-события.

2.4 Экспериментальная оценка и обоснование выбора конфигурации RAG

Для эмпирического обоснования выбранной конфигурации поиска (подраздел 2.2) была проведена сравнительная оценка пяти конфигураций на размеченном датасете `gold_sources.json` объёмом 54 описания сделок. Каждое описание сопровождается тройкой «тип договора, перечень источников, перечень спецификаций статей», представляющей собой эталонный ответ юриста-эксперта на вопрос «какие нормы релевантны для подготовки договора по этому описанию». Запуск выполнялся в режиме прерывания графа после узла поиска норм; уточняющие вопросы агента симулировались отдельным LLM-вызовом (роль «человек, описывающий сделку»), что снимает зависимость от ручного запуска. Для каждого прогона фиксировались решение классификатора, результаты поиска и измеренная задержка узла поиска.

Метрики определены следующим образом. Для каждого описания заданы множество эталонных спецификаций норм $G = \{g_1, \dots, g_{|G|}\}$ и упорядоченная последовательность извлечённых системой чанков $R = (r_1, r_2, \dots)$; чанк r_i считается релевантным, если его пара «источник, статья» попадает в какую-либо спецификацию g_j (с нормализацией по морфологии и регистру для имени источника и без учёта дробной части номера статьи). Обозначим через $\text{rel}(R, k)$ множество релевантных чанков в первых k позициях выдачи; тогда

$$\text{precision}@k = \frac{|\text{rel}(R, k)|}{k}, \quad (4)$$

$$\text{recall}@k = \frac{|\{g_j \in G \mid \exists r_i \in \text{rel}(R, k), r_i \text{ соответствует } g_j\}|}{|G|}, \quad (5)$$

$$\text{hit}@k = \mathbb{I}[|\text{rel}(R, k)| \geq 1], \quad (6)$$

$$\text{accuracy}_{\text{cls}} = \frac{1}{N} \sum_{n=1}^N \mathbb{I}[\hat{t}_n \equiv t_n], \quad (7)$$

где $\hat{t}_n$ — предсказанный тип сделки, t_n — эталонный тип, отношение $\equiv$ толерантно к скобочным уточнениям и регистру. Сравнивались пять конфигураций, представляющих разумные варианты двухступенчатого поиска

(во всех использовалась модель эмбедингов ai-sage/Giga-Embeddings-instruct в режиме bfloat16, различия касаются второго этапа — реранкера и LLM-фильтра): базовая (только бикодер); бикодер + реранкер BAAI/bge-reranker-v2-m3; бикодер + bge-reranker-v2-m3 + LLM-фильтр; бикодер + Qwen/Qwen3-Reranker-0.6B; бикодер + Qwen3-Reranker + LLM-фильтр (текущая). Результаты сведены в таблице 2.1.

Таблица 2.1 – Сравнение пяти конфигураций поиска (значения в процентах; latency — в миллисекундах)

Конфигурация	acc.	p@3	r@3	h@3	p@5	r@5	p@10	r@10	r@20	lat.
#1 base (без реранкера, без фильтра)	100	42.6	35.8	81.5	41.1	40.7	38.0	50.1	56.0	243
#2 base + bge-v2-m3	100	77.8	43.9	98.1	71.5	45.2	64.4	51.3	58.3	2050
#3 base + bge-v2-m3 + LLM-фильтр	100	85.8	43.8	98.1	82.2	46.8	69.3	55.7	58.6	9766
#4 base + Qwen3-Reranker-0.6B	100	85.8	43.5	98.1	81.5	46.9	70.2	51.2	58.6	4309
#5 base + Qwen3-Reranker + LLM-фильтр (текущая)	100	91.4	46.0	100	89.6	49.9	78.0	55.0	56.0	13644

Полученные значения метрик демонстрируют поступательное улучшение качества поиска с каждым дополнительным компонентом конвейера. Базовая конфигурация #1 (только бикодер) обеспечивает попадание релевантной нормы в первые три позиции выдачи в 81.5 % случаев. Подключение кросс-энкодерного реранкера (конфигурация #2) повышает precision@3 с 42.6 % до 77.8 % (рост в 1.83 раза), а hit@3 — с 81.5 % до 98.1 %, что согласуется с известным свойством двухступенчатого поиска: бикодер обеспечивает охват, кросс-энкодер дотягивает точность ранжирования верхушки выдачи. Дальнейшее подключение LLM-фильтра поверх классического реранкера BGE (конфигурация #3) даёт дополнительный точностный прирост (precision@3=85.8 %); замена реранкера на современный Qwen3-Reranker-0.6B без фильтра (конфигурация #4) обеспечивает аналогичный точностный уровень при меньшей латентности. Итоговая конфигурация #5, объединяющая Qwen3-Reranker с LLM-фильтром, достигает наилучших показателей по всем

ключевым метрикам ранжирования: $\text{hit}@3=100\%$, $\text{precision}@3=91.4\%$, $\text{precision}@5=89.6\%$, $\text{precision}@10=78.0\%$. Метрики итоговой конфигурации в детализированном виде по всем рассматриваемым значениям k приведены на рисунке 2.4.

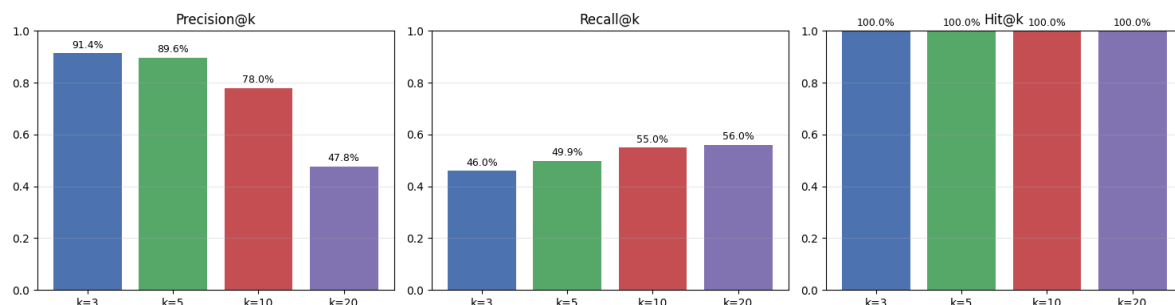


Рисунок 2.4 – Метрики итоговой конфигурации #5:
 $\text{precision}@k, \text{recall}@k, \text{hit}@k$ для $k \in \{3, 5, 10, 20\}$

Приведённые на рисунке 2.4 значения — усреднённые по всему набору из 54 описаний сделок. Распределение тех же метрик в разрезе типов договоров приведено на рисунке 2.5: точность классификации достигает значения 100 % для каждого из представленных типов сделок, оценки $\text{precision}@3$ сосредоточены в верхней зоне шкалы, а при росте k распределение значений ожидаемо смещается вниз — естественное следствие более строгого требования к попаданию релевантной нормы в большее число позиций выдачи.

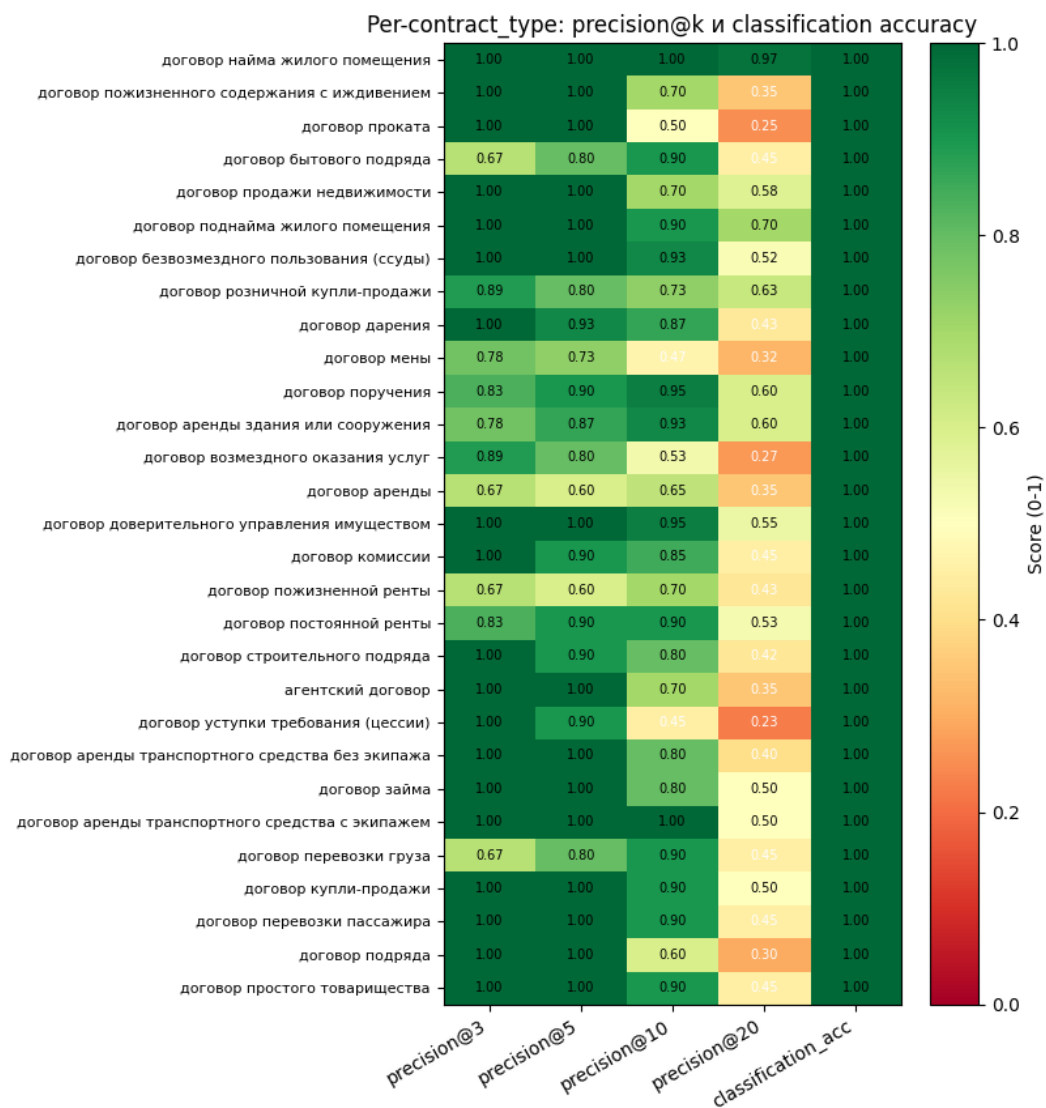


Рисунок 2.5 – Распределение значений precision@k и точности классификации итоговой конфигурации поиска по типам сделок

Точность классификации сделки во всех конфигурациях составляет 100 %, что объясняется надёжностью двухступенчатой стратегии классификатора (основной классификатор плюс субагент с RAG-поддержкой при низкой уверенности) на коротких словесных описаниях. Достигнутый уровень метрик соответствует целевой постановке задачи: стопроцентный hit@3 означает, что нижестоящие узлы графа гарантированно получают релевантную норму в составе своего контекста, а precision@3=91.4 % обеспечивает, что верхушка выдачи практически полностью состоит из применимых норм и не загрязняет вход LLM шумом. Замеренная задержка узла поиска в 13.6 с приемлема для целевого сценария подготовки договора, где пользователь ожидает результат на временной шкале десятков секунд. По

совокупности результатов в качестве рабочей конфигурации поиска принята
связка `ai-sage/Giga-Embeddings-instruct` +
`Qwen/Qwen3-Reranker-0.6B` + LLM-фильтр на батчах размера 10.

3 РЕЗУЛЬТАТЫ РАЗРАБОТКИ

В настоящей главе представлены результаты разработки: целевые условия и сценарии применения системы, демонстрация её работы на типовом пользовательском сценарии, итоги внешней экспертной оценки качества подготавливаемых артефактов и сводные технические характеристики решения.

3.1 Условия и сценарии практического применения разработанной системы

Разработанная система ориентирована на физических лиц и субъектов малого предпринимательства, заключающих типовые сделки гражданского оборота (подробное обоснование — в подразделе 1.1). Целевая предметная область — поименованные в ГК РФ сделки гражданского оборота, типичные для бытовой практики и для оборота малого бизнеса. Конкретный перечень поддерживаемых типов договоров не зафиксирован в исходном коде, а определяется составом проиндексированного корпуса нормативных правовых актов: администратор через панель управления загружает источники в индекс и регистрирует соответствующие типы сделок в каталоге без перекомпиляции приложения (механизм рассмотрен в подразделе 2.3). Тем самым система адаптивна по охвату: эксплуатирующая организация настраивает поддерживаемые типы сделок под собственный сценарий применения. Корпоративные сделки со сложным предметом, сделки с особым нотариальным режимом и сделки с публичными элементами выходят за пределы целевой постановки задачи.

Режим доступа — web-интерфейс с входом через внешних провайдеров идентификации (Google или Яндекс по протоколу OAuth 2.0). Тарификация двухуровневая: бесплатный план с количественным лимитом и без возможности редактирования сгенерированного документа; платные планы снимают эти ограничения. Оплата осуществляется через ЮKassa. Принципиальный режим эксплуатации — сохранение конфиденциальности данных пользователя: при локальном развёртывании архитектура не предполагает передачи сведений третьим лицам (см. подраздел 2.3).

3.2 Демонстрация работы агента в типовом сценарии

Типовой пользовательский сценарий охватывает последовательность шагов от описания сделки до получения готового пакета документов. После аутентификации пользователь формулирует словесное описание планируемой сделки в свободной форме (например: «хочу сдать однокомнатную квартиру в Москве на 11 месяцев, арендатор — физическое лицо»); сообщение поступает в эндпоинт `POST /api/chat` и обрабатывается графом состояний агента (подраздел 2.1). Если описание недостаточно для согласования существенных условий, узел проверки достаточности данных формирует уточняющий вопрос и приостанавливает граф через сохранение контрольного пункта; следующее сообщение пользователя возобновляет обработку с того же узла. Пример диалогового взаимодействия приведён на рисунке 3.1.

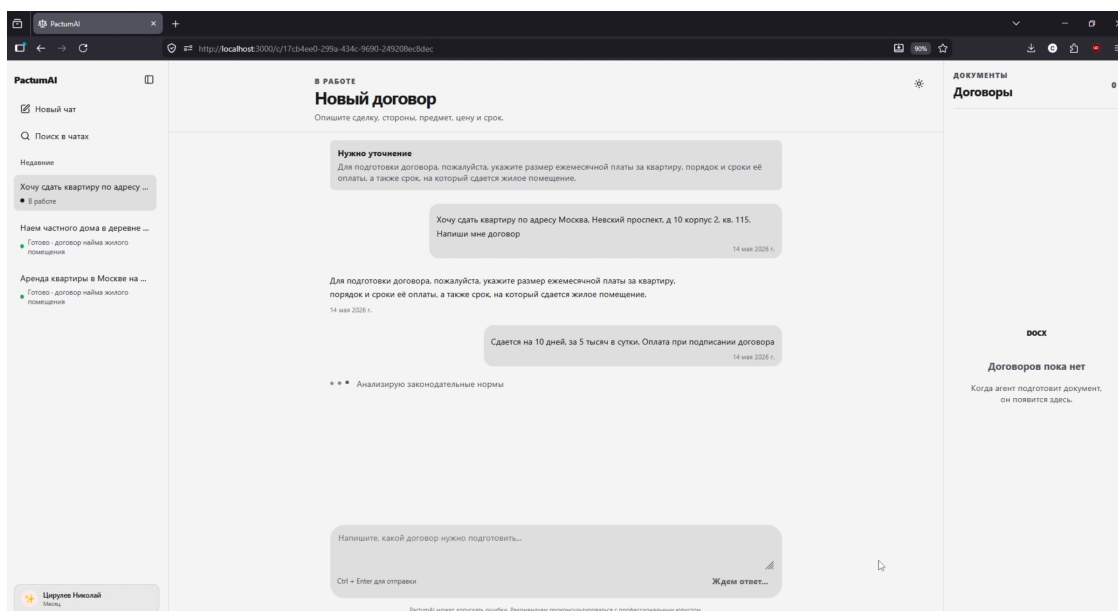


Рисунок 3.1 – Окно чата с типовым диалогом: запрос пользователя, уточняющий вопрос агента и ответ пользователя

После сбора существенных сведений граф последовательно выполняет проверку правовой допустимости сделки по общим нормам, поиск релевантных норм по корпусу нормативных правовых актов, генерацию проекта договора и его автоматическую итеративную валидацию (подразделы 2.1 и 2.2). Узел формирования рекомендаций выдаёт юридический отчёт со ссылками на статьи нормативных правовых актов в формате [Источник, статья], а узел сохранения результата конвертирует проект договора в DOCX-файл и публикует ссылку для скачивания. Окно чата

на этом этапе показано на рисунке 3.2.

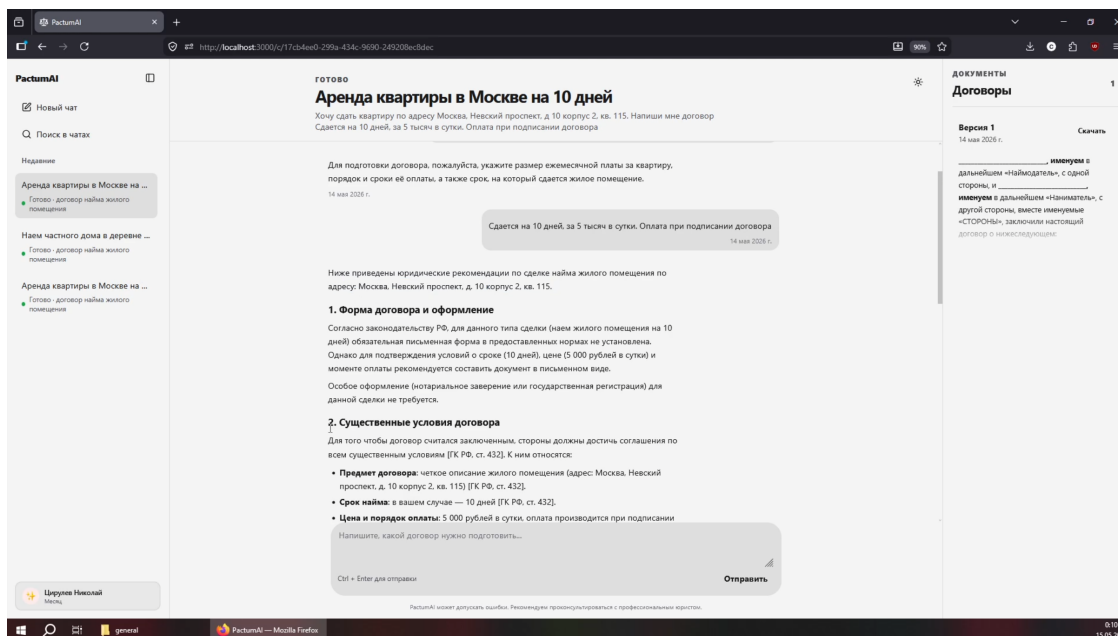


Рисунок 3.2 – Окно чата с готовыми юридическими рекомендациями и ссылкой на скачивание сгенерированного проекта договора

Сгенерированный проект оформлен в типовой структуре делового документа: реквизиты сторон, девять содержательных разделов (предмет; срок действия; права и обязанности сторон; цена и порядок расчётов; ответственность; изменение и расторжение; разрешение споров; заключительные положения; реквизиты и подписи) и поля для рукописного заполнения недостающих сведений; шрифт Times New Roman 12 пт, полуторный межстрочный интервал, поля по 25 мм. Пример страницы приведён на рисунке 3.3.

ДОГОВОР ПОСТОЯННОЙ РЕНТЫ

№ _____

г. _____

« ____ » _____ 20 ____ г.

Соколова Людмила Ивановна, паспорт серии 4516 № 384721, выдан ОМВД России по району Черёмушки г. Москвы 17.05.2016, код подразделения 770-054, адрес регистрации: 117418, г. Москва, ул. Новочерёмушкинская, д. 52, кв. 31, именуемая в дальнейшем «ПОЛУЧАТЕЛЬ РЕНТЫ», с одной стороны, и Орлов Максим Сергеевич, паспорт серии 4521 № 618452, выдан ОМВД России по району Раменки г. Москвы 11.02.2021, код подразделения 770-089, адрес регистрации: 119607, г. Москва, Мичуринский проспект, д. 21, кв. 77, именуемый в дальнейшем «ПЛАТЕЛЬЩИК РЕНТЫ», с другой стороны, вместе именуемые «СТОРОНЫ», заключили настоящий договор о нижеследующем:

1. ПРЕДМЕТ ДОГОВОРА

1.1. ПОЛУЧАТЕЛЬ РЕНТЫ передает ПЛАТЕЛЬЩИКУ РЕНТЫ в собственность двухэтажный коттедж площадью 77 квадратных метров, расположенный по адресу: Московская область, Истринский городской округ, коттеджный поселок «Лесные Озёра», ул. Сосновая, д. 14 (кадастровый номер: 50:08:0040217:3185).

1.2. В обмен на полученное имущество ПЛАТЕЛЬЩИК РЕНТЫ обязуется выплачивать ПОЛУЧАТЕЛЮ РЕНТЫ ренту в виде определенной денежной суммы.

2. СРОК ДЕЙСТВИЯ ДОГОВОРА

2.1. Обязанность по выплате ренты устанавливается бессрочно (постоянная рента).

3. ПРАВА И ОБЯЗАННОСТИ СТОРОН

3.1. ПОЛУЧАТЕЛЬ РЕНТЫ обязуется передать имущество в собственность ПЛАТЕЛЬЩИКА РЕНТЫ в сроки и на условиях, согласованных СТОРОНАМИ.

3.2. ПЛАТЕЛЬЩИК РЕНТЫ обязуется осуществлять периодические выплаты ренты в размере и сроки, установленные настоящим договором.

Рисунок 3.3 – Содержательная страница сгенерированного проекта договора в формате DOCX

Параллельно с пользовательским контуром в системе предусмотрен административный контур (рисунок 3.4): загрузка нормативных актов, контроль за результатом деления на чанки и индексация в нужной коллекции Qdrant, управление каталогом типов сделок, тарифными планами и административными учётными записями.

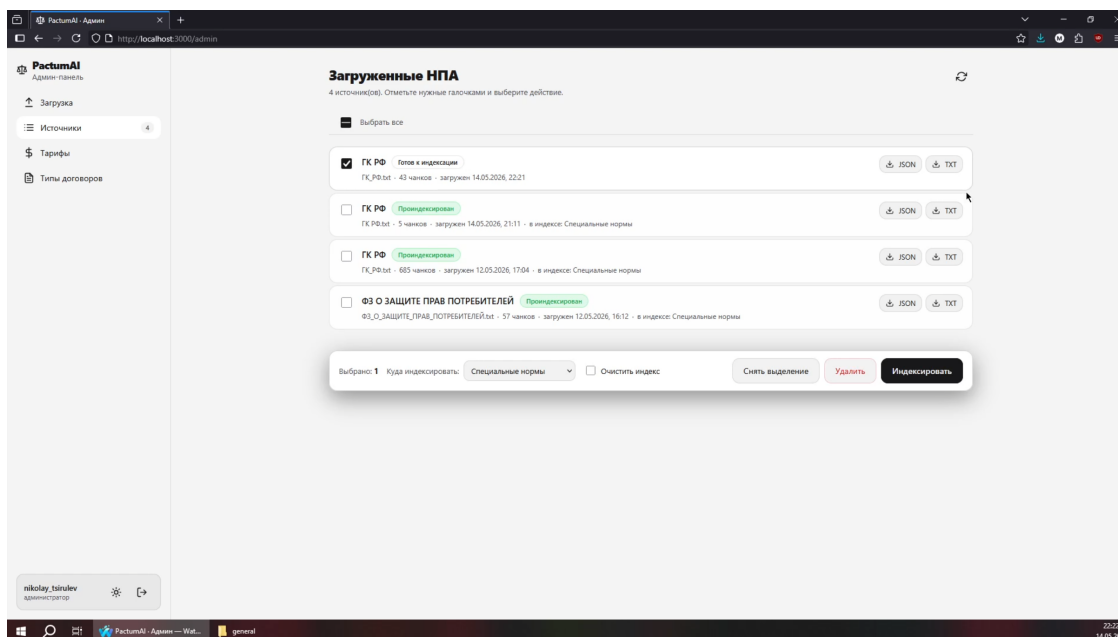


Рисунок 3.4 – Административная панель управления корпусом нормативных правовых актов

3.3 Экспертная оценка качества подготовленных артефактов

Для качественной оценки разработанной системы проведена внешняя экспертная апробация с привлечением трёх практикующих юристов. Все привлечённые эксперты в анкетных сведениях указали высшее юридическое образование, полученное в Московском государственном юридическом университете имени О.Е. Кутафина (МГЮА) по специальности «Юриспруденция». Постановка исследования: эксперту-юристу предъявляются готовые артефакты системы — сгенерированный проект договора и сопровождающие юридические рекомендации, — и эксперт независимо оценивает их по принятой шкале. Размер выборки соответствует характеру исследования: проведённая оценка имеет характер апробации, а не статистически достоверного экспертного исследования. Инструмент сбора оценок — web-форма опроса, размещённая на платформе `form.jotform.com`. На первой странице формы помещена преамбула об обезличенности полученных результатов и собираются анкетные сведения об эксперте (полученное образование, специальность и стаж практической работы в юридической сфере). На каждой из последующих страниц рассматривается один кейс: эксперту предъявляется пользовательское описание сделки, выданные системой юридические рекомендации и проект договора в формате DOCX, после чего по каждому артефакту выставляется

оценка по принятой шкале.

Шкала оценки — пятибалльная шкала Лайкерта [2] с подписанными якорями: 1 — критически неприемлемо; 2 — слабо; 3 — удовлетворительно; 4 — хорошо; 5 — отлично, можно использовать. Сбор оценок проводился по двум показателям — качество сгенерированного проекта договора и качество сопровождающих юридических рекомендаций. Массив кейсов составили одиннадцать модельных сделок, охватывающих представительный для бытовой практики набор поименованных в ГК РФ типов договоров (купля-продажа, продажа недвижимости, дарение, мена, заём, аренда, прокат, наём жилого помещения, подряд, возмездное оказание услуг, агентский договор); три эксперта независимо оценивали каждый кейс по двум показателям, что даёт массив из 66 оценок.

Полученные оценки полностью попадают в верхнюю половину шкалы — ни один артефакт не был оценён ниже уровня «хорошо». Распределение оценок приведено в таблице 3.1, средние — в таблице 3.2.

Таблица 3.1 – Распределение полученных оценок по показателям

Показатель	«4», шт.	«5», шт.	«4», %	«5», %
Качество проекта договора	19	14	57.6	42.4
Качество рекомендаций	9	24	27.3	72.7

Таблица 3.2 – Средние оценки по экспертам и показателям

Эксперт	Качество договора	Качество рекомендаций	Разница
Эксперт 1	4.36	4.55	0.19
Эксперт 2	4.45	4.73	0.28
Эксперт 3	4.45	4.91	0.46
В среднем	4.42	4.73	0.31

Средняя оценка качества проекта договора по выборке составила 4.42, средняя оценка качества рекомендаций — 4.73. Систематическая разница в пользу рекомендаций воспроизводится у каждого из трёх экспертов и объясняется содержательной разницей задач, решаемых двумя артефактами. Качество рекомендаций определяется прежде всего корректностью

идентификации применимых норм и сопровождающих сделку рисков; LLM в режиме генерации, дополненной поиском по проиндексированному корпусу, надёжно подсвечивает существенные условия и сопровождает каждое утверждение проверяемой ссылкой на статью нормативного акта, что и фиксируется экспертами в верхней части шкалы. Подготовка проекта договора предъявляет к системе дополнительное требование — воспроизведение традиционной для данного типа сделки структуры и формулировок, которая не закреплена в законе, а сложилась в правоприменительной практике. Большинство свободных комментариев экспертов по проекту договора фиксировало именно этот аспект: документ оценивался как юридически корректный, но менее привычный по форме, чем типовой образец, который применил бы практикующий юрист. Расширение библиотеки типовых шаблонов договоров позволит при сохранении единого ядра системы порождать документ ожидаемой для каждого типа сделки структуры без отступления от его юридической корректности; настоящая работа использует единый универсальный шаблон, и развитие библиотеки типовых шаблонов выделено в направление дальнейшей работы. Распределение оценок по типам сделок приведено на рисунках 3.5 и 3.6: оценки удерживаются в диапазоне $[4.0, 5.0]$ для всех одиннадцати рассмотренных типов договоров.

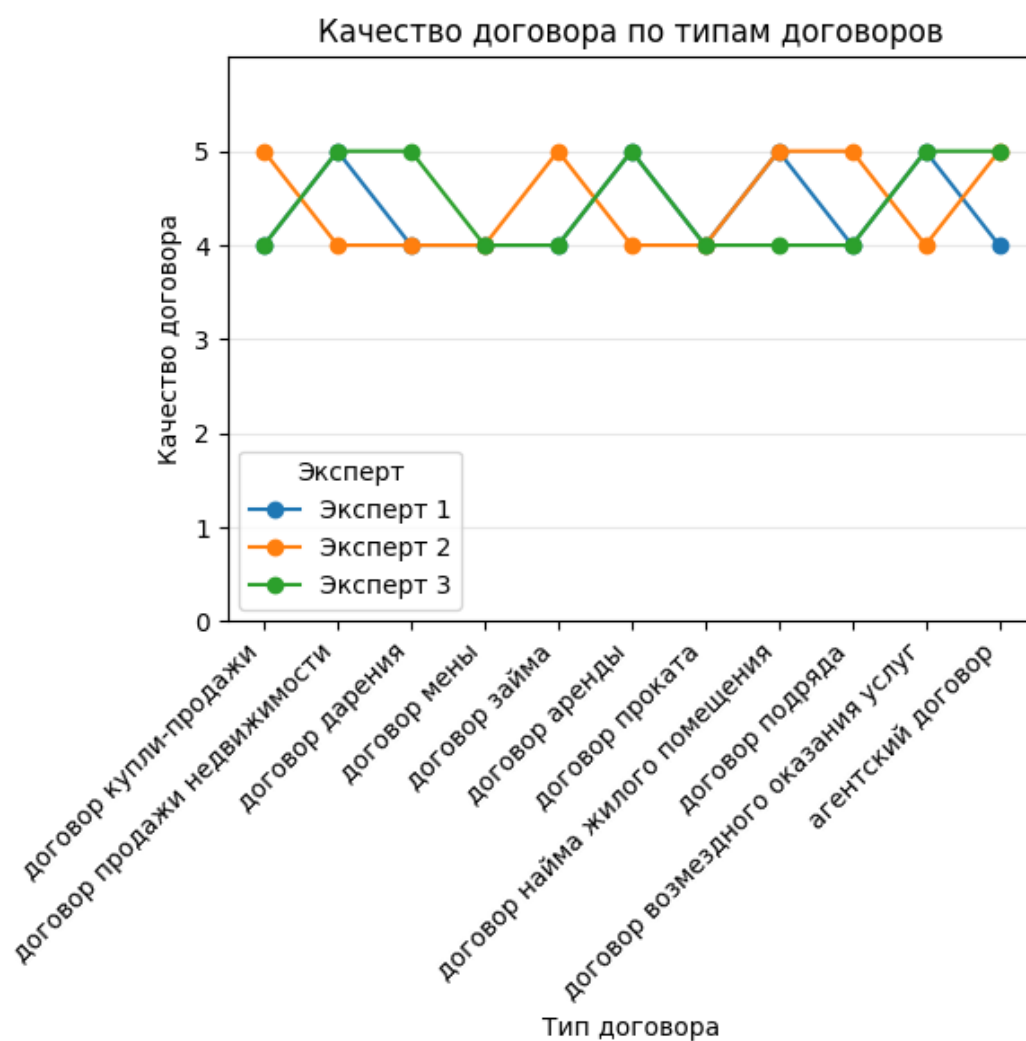


Рисунок 3.5 – Оценки качества проекта договора по типам договоров (одна линия — один эксперт)

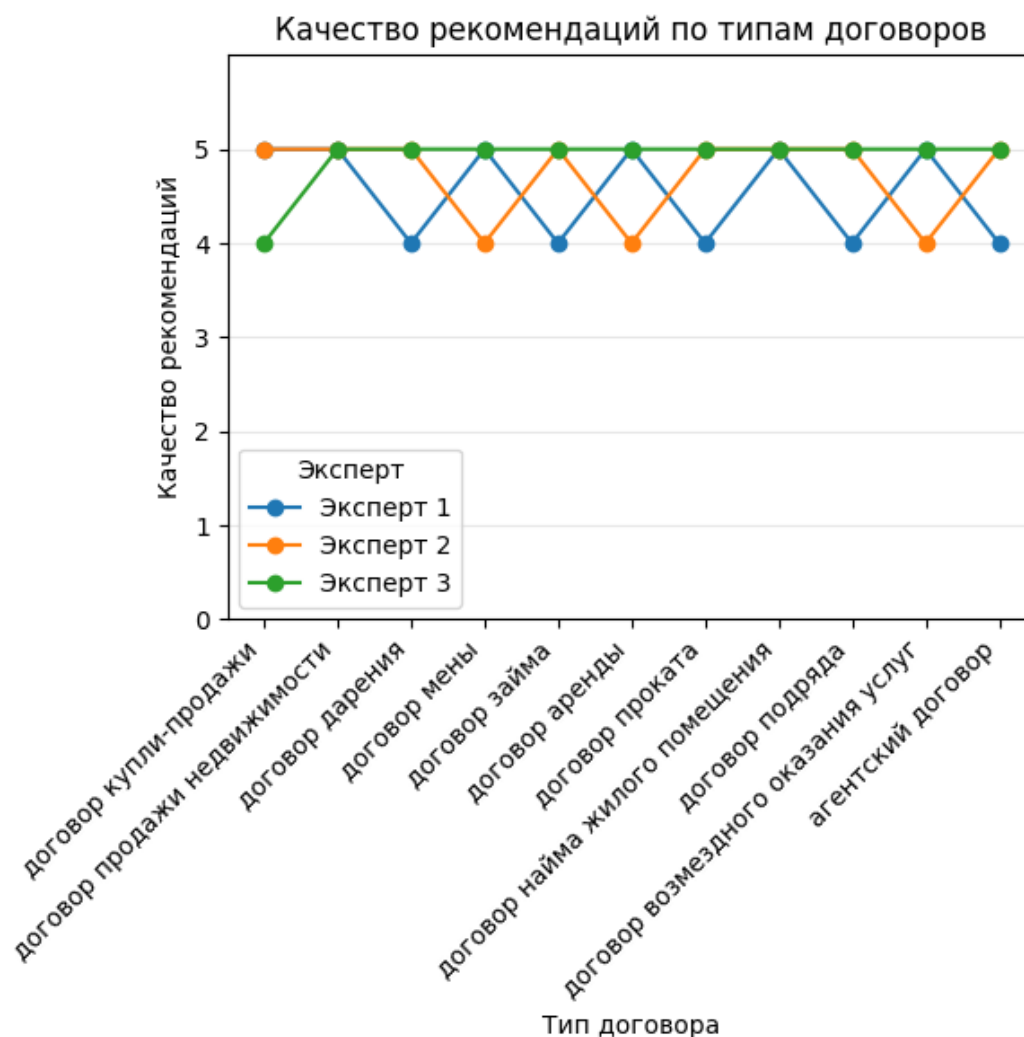


Рисунок 3.6 – Оценки качества рекомендаций по типам договоров (одна линия — один эксперт)

Ограничения проведённого исследования: выборка из трёх экспертов носит характер апробации; интегральная двухосевая оценка не разделяет ответственность за итоговый результат между подсистемами агента. Направления дальнейшего развития исследования — увеличение экспертной выборки и развёртывание оценки на несколько отдельных показателей (корректность ссылок на нормы права, полнота существенных условий, структурная корректность договора).

3.4 Технические характеристики разработанного решения

Сводные технические характеристики приведены в таблице 3.3. Производительность системы удерживается в рамках поставленного в подразделе 1.3 требования о среднем времени отклика в пределах единиц

минут на полный цикл подготовки документа; доминирующий вклад в задержку даёт поиск (около 13.6 с на узел `retrieve_norms`, см. подраздел 2.4). Корпус нормативных правовых актов в текущем развёртывании включает части первую и вторую ГК РФ и ряд профильных федеральных законов; совокупный объём — порядка нескольких тысяч чанков уровня отдельной статьи или её обособленного пункта. В режиме облачной маршрутизации Ollama локальные требования к видеопамяти ограничены моделями поиска (единицы гигабайт VRAM); в режиме полностью локального исполнения LLM пользовательские данные не покидают периметра эксплуатирующей организации (см. подраздел 2.3).

Таблица 3.3 – Сводные технические характеристики разработанного решения

Характеристика	Значение
Поддерживаемые типы договоров	поименованные в ГК РФ сделки; перечень настраивается администратором через панель управления
Число коллекций корпуса в Qdrant	3 (общие, основные, дополнительные нормы)
Число узлов графа состояний агента	21, объединены в семь функциональных фаз
Модель эмбедингов	Giga-Embeddings-instruct, размерность 2048
Реранкер	Qwen3-Reranker-0.6B
Метрики поиска	hit@3=100 %, precision@3=91.4 %
Средняя задержка узла поиска норм	13.6 с
Доступные интерфейсы	web-чат, WebSocket, REST API
Идентификация пользователей	Google, Яндекс (OAuth 2.0)
Платёжный шлюз	ЮKassa
Развёртывание	Docker Compose, 7 сервисов

3.5 Выводы по главе

В настоящей главе представлены результаты разработки. Зафиксирована целевая область применения системы — поименованные в ГК РФ сделки гражданского оборота, типичные для бытовой практики физических лиц и для оборота малого бизнеса; конкретный перечень поддерживаемых типов договоров определяется составом проиндексированного корпуса нормативных правовых актов и настраивается администратором без изменения исходного кода. Продемонстрирован типовой пользовательский сценарий взаимодействия с системой: от описания планируемой сделки в свободной форме до получения готового проекта договора в формате DOCX и сопровождающих юридических рекомендаций со ссылками на конкретные нормы права. Проведена внешняя экспертная апробация качества готовых артефактов; средние оценки по выборке (4.42 по проекту договора и 4.73 по рекомендациям из 5 возможных при отсутствии оценок ниже 4) подтверждают приемлемый уровень качества для целевого сценария бытовых сделок. Сведены технические характеристики решения, фиксирующие его соответствие поставленному в подразделе 1.3 техническому заданию.

ЗАКЛЮЧЕНИЕ

В настоящей выпускной квалификационной работе достигнута поставленная цель — разработан интеллектуальный программный агент, способный по произвольному словесному описанию сделки определять её гражданско-правовой тип, в диалоговом режиме собирать у пользователя недостающие для подготовки документа сведения и формировать проект договора в формате DOCX вместе с юридическими рекомендациями со ссылками на конкретные статьи нормативных правовых актов. Все поставленные во введении задачи решены, работа выполнена в полном объёме.

Сформирован корпус нормативных правовых актов российского договорного права, включающий часть первую и часть вторую Гражданского кодекса Российской Федерации и ряд профильных федеральных законов; реализованы предобработка корпуса, его разбиение на чанки уровня отдельной статьи или её обособленного пункта и индексация в векторной базе данных Qdrant в трёх коллекциях, разнесённых по уровню обобщённости норм. Спроектирована формальная модель агента в виде ориентированного графа состояний из двадцати одного узла, объединённых в семь функциональных фаз обработки запроса; модель реализована программно на фреймворке LangGraph с поддержкой прерывания обработки на уточнения у пользователя и последующего возобновления с восстановлением промежуточного состояния из контрольного пункта. Реализован механизм диалогового уточнения недостающих сведений: при нехватке данных для согласования существенных условий договора агент формулирует пользователю целевой уточняющий вопрос и продолжает обработку после получения ответа без повторного исполнения предшествующих шагов. Реализована автоматическая итеративная проверка сгенерированного проекта договора на полноту существенных условий и непротиворечивость применимым нормам с возможностью повторной генерации при обнаружении замечаний. Разработан модуль генерации файла договора в формате DOCX с надлежащим оформлением документа делового стиля. Разработаны веб-интерфейс пользователя с двунаправленными уведомлениями через WebSocket и панель администратора для управления корпусом нормативных правовых актов, каталогом поддерживаемых типов сделок и тарифными

планами. Обеспечено контейнерное развёртывание программной системы средствами Docker Compose и её работоспособность на локальном оборудовании без передачи пользовательских данных во внешние сервисы. Проведена внешняя экспертная оценка качества подготовленных системой артефактов с привлечением практикующих юристов.

Полученные количественные результаты подтверждают работоспособность принятых проектных решений. Эмпирическая оценка поиска на специально подготовленном датасете из 54 эталонных описаний сделок показала, что в итоговой конфигурации (модель эмбедингов Giga-Embeddings-instruct, перанкер Qwen3-Reranker-0.6B, LLM-фильтр) достигаются метрики верхней части выдачи $hit@3=100\%$ и $precision@3=91.4\%$ при точности классификации типа сделки 100% ; средняя задержка узла поиска составила 13.6 с, что удерживает полный цикл подготовки документа в рамках поставленного техническим заданием требования к среднему времени отклика. Внешняя экспертная апробация подготовленных артефактов с привлечением трёх практикующих юристов показала, что средняя оценка качества проекта договора по пятибалльной шкале Лайкерта составила 4.42, средняя оценка качества юридических рекомендаций — 4.73 при отсутствии оценок ниже уровня «хорошо» по обоим показателям и для всех одиннадцати рассмотренных типов сделок. Эти результаты подтверждают практическую применимость предложенного подхода для подготовки договоров по бытовым сделкам гражданского оборота для пользователей без юридического образования.

Научная новизна работы состоит в построении интеллектуального агента, объединяющего в одной программной системе три свойства: работу с произвольной формулировкой сделки от пользователя, опору на проиндексированный корпус российского законодательства и автоматический контроль качества подготавливаемого проекта договора. Подобное сочетание свойств на момент выполнения настоящей работы в открытой литературе и в практических программных продуктах в области правовых технологий не зафиксировано. Практическая значимость работы состоит в том, что разработанная система реализована в виде целостного программного продукта, адаптивного по охвату (перечень поддерживаемых типов сделок настраивается администратором без изменения исходного кода) и сохраняющего конфиденциальность пользовательских данных при локальном

развёртывании.

Ограничения проведённого исследования сформулированы открыто и определяют направления дальнейшего развития работы. Выборка из трёх экспертов носит характер апробации; целесообразно увеличение выборки до уровня статистической достоверности экспертного исследования и развёртывание интегральной оценки на несколько отдельных показателей качества (корректность ссылок на нормы права, полнота существенных условий, структурная корректность договора, полнота указания рисков и формальных требований). Дальнейшее развитие системы предполагает расширение каталога поддерживаемых типов сделок за счёт менее распространённых поименованных в Гражданском кодексе Российской Федерации договоров и сопровождение его соответствующим расширением проиндексированного корпуса нормативных правовых актов; расширение библиотеки типовых шаблонов договоров для воспроизведения сложившейся в правоприменительной практике структуры и формулировок документа без отступления от его юридической корректности; интеграцию с государственными информационными системами для автоматического предзаполнения реквизитов сторон по согласию пользователя; развитие поиска за счёт учёта актов толкования права (постановлений и обзоров судебной практики высших судебных инстанций) для повышения качества юридических рекомендаций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis [и др.]. — 2020. — DOI: 10.48550/arXiv.2005.11401. — arXiv: 2005.11401. — URL: <https://arxiv.org/abs/2005.11401> (дата обращения 14.05.2026).
2. Likert R. A Technique for the Measurement of Attitudes // Archives of Psychology. — 1932. — Т. 140. — С. 1—55.
3. Гражданский кодекс Российской Федерации (часть первая) от 30.11.1994 № 51-ФЗ (ред. от 26.03.2026). — 1994. — Собрание законодательства Российской Федерации. 1994. № 32. Ст. 3301.
4. Гражданский кодекс Российской Федерации (часть вторая) от 26.01.1996 № 14-ФЗ (ред. от 16.12.2025). — 1996. — Собрание законодательства Российской Федерации. 1996. № 5. Ст. 410.
5. Jurafsky D., Martin J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models. — 3-е изд. — 2024. — URL: <https://web.stanford.edu/~jurafsky/slp3/> (дата обращения 19.05.2026) ; Online draft, latest update 2026-01-06.
6. Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models / M. Dahl [и др.] // Journal of Legal Analysis. — 2024. — Т. 16, № 1. — С. 64—93. — DOI: 10.1093/jla/laaе003.
7. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models / N. Guha [и др.] // Advances in Neural Information Processing Systems. Т. 36. — 2023. — DOI: 10.48550/arXiv.2308.11462. — arXiv: 2308.11462. — URL: <https://arxiv.org/abs/2308.11462> (дата обращения 14.05.2026).
8. Retrieval-Augmented Generation for Large Language Models: A Survey / Y. Gao [и др.]. — 2023. — DOI: 10.48550/arXiv.2312.10997. — arXiv: 2312.10997. — URL: <https://arxiv.org/abs/2312.10997> (дата обращения 14.05.2026).

9. Accurate AI Assistance in Contract Law Using Retrieval-Augmented Generation to Advance Legal Technology / Y. Amazou [и др.] // International Journal of Advanced Computer Science and Applications. — 2025. — Т. 16, № 2. — С. 1141—1150. — DOI: 10.14569/IJACSA.2025.01602113.
10. A survey on large language model based autonomous agents / L. Wang [и др.] // Frontiers of Computer Science. — 2024. — Т. 18, № 6. — С. 186345. — DOI: 10.1007/s11704-024-40231-1.
11. LangChain AI. LangGraph Documentation. — 2026. — URL: <https://langchain-ai.github.io/langgraph/> (дата обращения 16.05.2026).
12. MinIO, Inc. MinIO Object Storage — Documentation. — 2026. — URL: <https://min.io/docs/minio/linux/index.html> (дата обращения 16.05.2026).
13. AI Sage. Giga-Embeddings-instruct: модель эмбедингов на основе GigaChat-3B. — 2025. — URL: <https://huggingface.co/ai-sage/Giga-Embeddings-instruct> (дата обращения 16.05.2026).
14. The Russian-Focused Embedders' Exploration: ruMTEB Benchmark and Russian Embedding Model Design / A. Snegirev [и др.] // Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). — Association for Computational Linguistics, 2025. — С. 236—254. — DOI: 10.18653/v1/2025.naacl-long.12. — URL: <https://aclanthology.org/2025.naacl-long.12> (дата обращения 16.05.2026).
15. Manning C. D., Raghavan P., Schütze H. Introduction to Information Retrieval. — Cambridge : Cambridge University Press, 2008. — ISBN 978-0-521-86571-5. — URL: <https://nlp.stanford.edu/IR-book/> (дата обращения 19.05.2026).
16. Qdrant. Reranking in Semantic Search — Qdrant documentation. — 2026. — URL: <https://qdrant.tech/documentation/search-precision/reranking-semantic-search/> (дата обращения 16.05.2026).
17. Goodfellow I., Bengio Y., Courville A. Deep Learning. — Cambridge, MA : MIT Press, 2016. — ISBN 978-0-262-03561-3. — URL: <https://www.deeplearningbook.org/> (дата обращения 19.05.2026).

18. Malkov Y. A., Yashunin D. A. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2020. — Т. 42, № 4. — С. 824—836. — DOI: 10.1109/TPAMI.2018.2889473.

19. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation / J. Chen [и др.] // Findings of the Association for Computational Linguistics: ACL 2024. — Association for Computational Linguistics, 2024. — DOI: 10.48550/arXiv.2402.03216. — arXiv: 2402.03216. — URL: <https://arxiv.org/abs/2402.03216> (дата обращения 16.05.2026).

20. Qwen Team. Qwen3-Reranker-0.6B: Model Card. — 2025. — URL: <https://huggingface.co/Qwen/Qwen3-Reranker-0.6B> (дата обращения 16.05.2026).

21. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena / L. Zheng [и др.] // Advances in Neural Information Processing Systems 36: Datasets and Benchmarks Track. — 2023. — DOI: 10.48550/arXiv.2306.05685. — arXiv: 2306.05685. — URL: <https://arxiv.org/abs/2306.05685> (дата обращения 16.05.2026).

22. Van Steen M., Tanenbaum A. S. Distributed Systems. — 4-е изд. — 2023. — ISBN 978-90-815406-3-6. — URL: <https://www.distributed-systems.net/index.php/books/ds4/> (дата обращения 19.05.2026).

23. Docker, Inc. Docker Compose Documentation. — 2026. — URL: <https://docs.docker.com/compose/> (дата обращения 16.05.2026).

24. Ramírez S. FastAPI Documentation. — 2026. — URL: <https://fastapi.tiangolo.com/> (дата обращения 16.05.2026).

25. Hardt D. The OAuth 2.0 Authorization Framework / Internet Engineering Task Force. — 2012. — URL: <https://www.rfc-editor.org/rfc/rfc6749> (дата обращения 16.05.2026).

ПРИЛОЖЕНИЕ А

Состояние агента

В таблице А.1 приведена полная спецификация полей типизированного состояния агента, определённого в модуле `state.py`. Состояние представляет собой словарь, наследующий стандартный `MessagesState` из библиотеки `LangGraph`; все собственные поля помечены признаком необязательного присутствия (`total=False`), что отражает характер их заполнения: каждое поле проставляется одним из узлов графа в свой момент времени, остальные узлы либо читают его, либо игнорируют. Поля сгруппированы по фазе обработки: входной запрос и классификация, поиск, проверки, выходные артефакты, контекст пользователя, субагент обработки дополнительных вопросов.

Таблица А.1 – Поля состояния ContractAgentState

Поле	Тип	Назначение
case_id	str	Идентификатор кейса, одновременно идентификатор потока в LangGraph
deal_description	str	Исходное словесное описание сделки от пользователя
deal_type	str None	Гражданско-правовой тип сделки после классификации
deal_type_confidence	str None	Уровень уверенности классификатора: low, medium, high
deal_classify_confidence	str None	Дополнительная оценка уверенности субагента классификации
classification_clarification_attempts	int	Счётчик попыток уточнения типа сделки
max_classification_clarifications	int	Жёсткий лимит на число уточнений классификации
clarification_stage	str None	Текущая стадия запроса уточнения: classification, data_sufficiency или optional_contract_generation
deal_structure	dict None	Структурированное представление сделки в виде словаря «поле:значение»
retrieval_query	str None	Запрос для поиска специальных норм
general_norms	list[dict] None	Чанки общих норм, загруженные в начале обработки
retrieved_norms	list[dict] None	Чанки специальных норм
general_check_passed	bool	Признак прохождения проверки общих норм договорного права
general_check_issues	list[str] None	Перечень выявленных правовых препятствий
general_check_explanation	str None	Пояснение для пользователя при выявленном препятствии
clarification_needed	bool	Признак необходимости запросить уточнение у пользователя
clarification_question	str None	Текст уточняющего вопроса
missing_fields	list[str] None	Список недостающих полей для подготовки договора
requires_written_form	bool None	Признак необходимости письменной формы для сделки
written_form_reason	str None	Краткое обоснование требования письменной формы
generate_optional_contract	bool None	Решение пользователя о необходимости генерировать проект договора
contract_md	str None	Текстовое превью сгенерированного договора
contract_html	str None	HTML-разметка сгенерированного договора
recommendations	str None	Юридические рекомендации в формате Markdown
validation_errors	list[str] None	Замечания валидации текущей итерации
iteration_count	int	Текущая итерация цикла регенерации
max_iterations	int	Жёсткий лимит на число итераций цикла регенерации
result_docx_path	str None	Путь к итоговому DOCX-файлу (S3-ключ или локальный путь)
result_saved	bool	Признак завершения сохранения результата
processing_stage	str None	Локализованное название текущей фазы обработки для WebSocket-уведомлений
intent	str None	Намерение пользователя: regenerate, edit или followup
contract_valid	bool None	Признак валидности проекта по результатам узла validate_contract
edit_summary	str None	Краткое описание правки от узла edit_contract
edit_changed_sections	list[str] None	Список разделов, затронутых точечной правкой
user_plan	str None	Код активного тарифного плана, инжектируется до запуска графа
allow_edit	bool	Биллинговый признак разрешения редактирования

Продолжение таблицы А.1

Поле	Тип	Назначение
contract_generation_policy	str None	Политика подготовки документа: legal_only, always, always_ask
ask_personal_data	bool	Пользовательская настройка: спрашивать ли персональные данные
final_contract_exchange	dict None	Стэшированный последний обмен LLM для дампа после валидации
final_recommendations_exchange	dict None	Стэшированный последний обмен LLM для рекомендаций
retrieve_norms_latency_ms	float None	Измеренная задержка узла retrieve_norms
subagent_active	bool	Признак активности субагента дополнительных вопросов
subagent_brief	str None	Бриф для субагента обработки дополнительных вопросов
subagent_user_question	str None	Вопрос пользователя, переданный субагенту
subagent_query	str None	Текущая формулировка поискового запроса субагента
subagent_query_history	list[str] None	Накопленный список уже использованных формулировок
subagent_attempts	int	Счётчик попыток субагента обработки дополнительных вопросов
subagent_max_attempts	int	Жёсткий лимит на число попыток субагента обработки дополнительных вопросов

Поле `messages` наследуется из `MessagesState` и содержит историю диалога пользователя с агентом в формате `LangChain BaseMessage`. Поля `user_plan`, `allow_edit`, `contract_generation_policy`, `ask_personal_data`, `max_iterations`, `max_classification_clarifications` инжектируются в начальное состояние сервером FastAPI из переменных окружения и активной подписки пользователя в момент запуска прохода и не изменяются в его пределах.

ПРИЛОЖЕНИЕ Б

Полный перечень узлов графа

В таблице Б.1 приведена развёрнутая таблица всех узлов графа, реализованных в модуле `nodes.py` и собранных в граф в модуле `graph.py`. Для каждого узла указаны: фаза обработки, ключевые читаемые из состояния поля (вход), записываемые в состояние поля (выход), используемые промпты из модуля `prompts.py`, а также внешние сервисы, к которым узел обращается. Полные тексты промптов, перечисленных в столбце «Промпт», приведены в приложении В.

Таблица Б.1 – Узлы графа состояний агента

Узел	Фаза	Входы	Выходы	Промпт	Сервисы
route_user_message	маршрутизация	messages	intent, processing_stage	CLASSIFY_FOLLOWUP_INTENT_PROMPT	LLM
gate_free_plan	биллинговая преграда	allow_edit, intent	messages, processing_stage	—	PostgreSQL
load_general_norms	загрузка корпуса	—	general_norms	—	Qdrant
classify_deal	классификация	deal_description, general_norms	deal_type, deal_structure, deal_type_confidence, clarification_needed	build_classify_deal_prompt, build_classify_rag_assist_prompt	LLM, Qdrant
inform_unsupported_deal	вспомогательный	deal_description	messages	build_inform_unsupported_deal_prompt	LLM
check_general_norms	проверка	deal_description, general_norms, deal_type	general_check_passed, general_check_issues, general_check_explanation	CHECK_GENERAL_NORMS_PROMPT	LLM
inform_user	вспомогательный	general_check_issues, general_check_explanation	messages	INFORM_USER_PROMPT	LLM
retrieve_norms	RAG-поиск	deal_type, deal_description, retrieval_query	retrieved_norms, retrieve_norms_latency_ms	BUILD_RETRIEVAL_QUERY_PROMPT, FILTER_RETRIEVED_NORMS_PROMPT; при включённом итеративном пути — ITERATIVE_RETRIEVAL_PLANNER_PROMPT, ITERATIVE_RELEVANCE_JUDGE_PROMPT, DETECT_VAGUE_REFERENCES_IN_NORMS_PROMPT	LLM, Qdrant, перанкер
check_written_form	проверка	retrieved_norms, deal_type, contract_generation_policy	requires_written_form, written_form_reason, clarification_stage	CHECK_WRITTEN_FORM_PROMPT, CLASSIFY_YESNO_PROMPT	LLM
check_data_sufficiency	проверка	deal_description, retrieved_norms, validation_errors, ask_personal_data	missing_fields, clarification_question, deal_structure	build_check_data_sufficiency_prompt	LLM
ask_clarification	прерывание	clarification_question, clarification_stage	—	—	—

Продолжение таблицы Б.1

Узел	Фаза	Входы	Выходы	Промпт	Сервисы
optional_contract_answer	вспомогательный	messages	generate_optional_contract, clarification_stage	CLASSIFY_YESNO_PROMPT	LLM
generate_contract	генерация	deal_type, deal_description, deal_structure, general_norms, retrieved_norms, validation_errors	contract_html, contract_md,final_contract_exchange	GENERATE_CONTRACT_PROMPT	LLM
edit_contract	точечная правка	contract_html, messages	contract_html, edit_summary, edit_changed_sections	EDIT_CONTRACT_PROMPT	LLM
validate_contract	LLM-судья	contract_html, deal_type, retrieved_norms	contract_valid, validation_errors	VALIDATE_CONTRACT_PROMPT	LLM
handle_validation_error	цикл регенерации	iteration_count	iteration_count	—	—
generate_recommendations	рекомендации	deal_description, deal_type, general_norms, retrieved_norms	recommendations, final_recommendations_exchange	GENERATE_RECOMMENDATIONS_PROMPT	LLM
enrich_recommendations	обогащение	recommendations, deal_type	recommendations (перезаписывается)	DETECT_VAGUE_REFERENCES_PROMPT, RAG_SUBAGENT_PROMPT, INTEGRATE_ENRICHMENTS_PROMPT	LLM, Qdrant
save_result	сохранение	contract_html, recommendations, case_id	result_docx_path, result_saved	—	MinIO, PostgreSQL, WebSocket
followup_response	доп. вопросы	messages, retrieved_norms, contract_md, recommendations	messages, поля subagent_*, subagent_active	FOLLOWUP_TRIAGE_PROMPT, FOLLOWUP_RESPONSE_PROMPT	LLM
followup_subagent_search	субагент доп. вопросов	subagent_brief, subagent_query, subagent_query_history	поля subagent_*, messages при found	FOLLOWUP_SUBAGENT_PROMPT	LLM, Qdrant

Условные функции маршрутизации между узлами реализованы в модулях `graph.py` и `routing.py`: `_route_after_user_message`, `_route_after_classification`, `_route_after_general_norms`, `_route_after_written_form`, `_route_after_data_sufficiency`, `_route_after_optional_contract_answer`, `_route_after_validation`, `_route_after_followup`, `_route_after_followup_subagent`. Каждая возвращает имя следующего узла строго из закрытого множества, перечисленного в декларации соответствующего вызова `add_conditional_edges` в сборке графа.

ПРИЛОЖЕНИЕ В

Ключевые промpts узлов графа и схема их сборки

Модуль `prompts.py` содержит порядка двадцати задачных промpts — по одному (а иногда нескольким) на каждый содержательный узел графа. Все промpts следуют одному шаблону: системный пролог `LEGAL_ASSISTANT_SYSTEM_PROMPT`, задаваемая узлу инструкция с явно перечисленными правилами и (для большинства узлов) точная спецификация JSON-ответа, который модель должна вернуть в режиме структурированного вывода. Типовой пример промpta и схема его сборки в узле графа приведены ниже.

Системный пролог `LEGAL_ASSISTANT_SYSTEM_PROMPT` (рисунок В.1) подключается ко всем содержательным вызовам языковой модели и задаёт общую роль «юрист, специализирующийся на российском гражданском праве», формат упоминания источников и запрет на выдуманные факты.

```
Ты
-- юрист, специализирующийся на российском гражданском праве. Тебя
зовут RastumAI. Источники
информации в каждом запросе:
- описание сделки и история диалога -- это то, что сообщил
  пользователь;
- разделы "Общие" нормы", "Специальные" нормы", "Нормы"" -- это
  выдержки из
  законодательства РФ, автоматически найденные в базе нормативных
  актов;
  пользователь их не присылал и не видит. Не
  используй формулировки в "предоставленных материалах", вы"
  предоставили нормы
  ", вы" дали ссылки" -- нормы взяты из законодательства РФ.
  Используй формулировки
  согласно законодательству РФ", в соответствии с источником [статья
  ]". Отвечай
  только на основе предоставленных норм и описания сделки. Не
  выдумывай факты.
```

Рисунок В.1 – Системный промпт
LEGAL_ASSISTANT_SYSTEM_PROMPT

В качестве содержательного примера приведён промпт генерации проекта договора GENERATE_CONTRACT_PROMPT (рисунок В.2) — наиболее ответственного узла графа, формирующего основной артефакт системы. Промпт включает HTML-шаблон делового документа с фиксированной структурой из девяти разделов, явно перечисляет разрешённые HTML-теги (body, p, strong, table, tr, td), запрещает использование Markdown и пояснений вне разметки и фиксирует ключевое правило «в самом тексте договора не упоминать статьи закона»: формулировки условий пишутся обычным языком, а соответствующие нормы и риски выносятся в раздел рекомендаций отдельным узлом.

Сгенерируй

только HTML-тело договора: от `<body>` до `</body>`. Договор должен соответствовать стандартному шаблону по оформлению (9 разделов: предмет, срок, права и обязанности, цена, ответственность, изменение/расторжение, порядок разрешения споров, заключительные положения, реквизиты и подписи). Обязательно добавь в договор поля для каждой из сторон: ФИО физлица(), ОГРНИПИНН / ИП(), полное наименование ОГРНИНН// юрлица(), адрес. Правила:

- Не выдумывай факты и реквизиты.
- Для недостающих данных используй только печатные поля: "_____", "20__".
- Разрешены только теги: `body`, `p`, `strong`, `table`, `tr`, `td`.
- Без markdown/json-пояснений/.
- Не указывай в тексте договора ссылки на статьи закона, кодексы или иные нормативные акты. Формулировки условий пиши обычным языком.

Рисунок В.2 – Промпт генерации проекта договора
GENERATE_CONTRACT_PROMPT

Схема сборки промпта в узле графа единообразна и иллюстрируется на примере узла `check_written_form` (рисунок В.3). Системный пролог `LEGAL_ASSISTANT_SYSTEM_PROMPT` и инструкция узла `CHECK_WRITTEN_FORM_PROMPT` являются константами; в момент вызова к ним конкатенацией добавляется человеческий промпт, содержащий релевантные поля состояния агента — историю диалога, тип сделки, исходное описание, найденные нормы. Названия секций («История диалога:», «Тип сделки:», «Описание сделки (от пользователя):», «Нормы (выдержки из законодательства РФ):») вынесены в отдельные константы, что обеспечивает единообразие между промптами и согласованность с инструкцией системного пролога о способах ссылок на содержимое разделов. Вызов `_invoke_json`

обращается к LLM в режиме структурированного вывода с предопределённой fallback-схемой на случай ошибки парсинга и возвращает уже разобранный объект, готовый к использованию в логике маршрутизации.

```
def check_written_form(state: ContractAgentState) -> dict:
    fallback = {"requires_written_form": True,
                "reason": письменная форма предполагается по
    умолчанию"}
    result = _invoke_json(
        "check_written_form",
        CHECK_WRITTEN_FORM_PROMPT, #
        системный + инструкция узла
        ( #
        человеческий промпт:
            f"{SECTION_LABEL_HISTORY}\n" # -
        история диалога
            f"{_conversation_context(state)}\n\n"
            f"{SECTION_LABEL_DEAL_TYPE} " # - тип
        сделки
            f"{state.get('deal_type') or '-'}\n\n"
            f"{SECTION_LABEL_DEAL_DESCRIPTION}\n" # -
        описание от пользователя
            f"{state.get('deal_description', '')}\n\n"
            f"{SECTION_LABEL_NORMS}\n" # -
        найденные нормы
            f"{_norms_to_context(state.get('retrieved_norms'))}"
        ),
        fallback=fallback,
    )
    ...
```

Рисунок В.3 – Схема сборки промпта в узле графа (фрагмент функции `check_written_form` из модуля `nodes.py`)

Остальные содержательные промпты модуля `prompts.py` построены по той же схеме и решают аналогичные задачи в своих узлах графа: `build_classify_deal_prompt` (классификация типа сделки), `CHECK_GENERAL_NORMS_PROMPT` (проверка препятствий по общим нормам), `BUILD_RETRIEVAL_QUERY_PROMPT` (формирование поискового

запроса для подсистемы RAG), `FILTER_RETRIEVED_NORMS_PROMPT` (LLM-фильтр выдачи), `CHECK_WRITTEN_FORM_PROMPT` (необходимость письменной формы), `build_check_data_sufficiency_prompt` (достаточность данных), `VALIDATE_CONTRACT_PROMPT` (валидация проекта договора), `GENERATE_RECOMMENDATIONS_PROMPT` (формирование юридического отчёта), `DETECT_VAGUE_REFERENCES_IN_NORMS_PROMPT` и `INTEGRATE_ENRICHMENTS_PROMPT` (обработка бланкетных отсылок), `EDIT_CONTRACT_PROMPT` (точечная правка договора), `CLASSIFY_FOLLOWUP_INTENT_PROMPT` и `FOLLOWUP_TRIAGE_PROMPT` (маршрутизация дополнительных вопросов), `FOLLOWUP_SUBAGENT_PROMPT` (изолированный поиск субагента), `CLASSIFY_YESNO_PROMPT` (классификация ответа «да/нет»), `build_inform_unsupported_deal_prompt` и `INFORM_USER_PROMPT` (формирование вежливых сообщений об отказе), `GENERATE_CASE_TITLE_PROMPT` (формирование заголовка кейса для интерфейса). Тексты промптов хранятся в модуле `app/agent/prompts.py` и снабжены подробными встроенными комментариями, поясняющими каждое правило.

ПРИЛОЖЕНИЕ Г

Граф состояний интеллектуального агента

Граф состояний интеллектуального агента, реализованный в модуле `graph.py`, содержит двадцать один узел, объединённый в семь функциональных фаз, и все условные переходы между ними. Полная схема графа приведена на рисунке Г.1.

Условные обозначения. Узлы графа изображены прямоугольниками со скруглёнными углами и подписаны идентификатором в исходном коде и кратким русским описанием выполняемой операции. Терминальные состояния обозначены кругами: заполненный чёрный круг — начальное состояние (входящее сообщение пользователя), круг с точкой внутри — допустимое конечное состояние (любой из возможных ответов пользователю). Узел `ask_clarification` выделен формой «заметка»: он приостанавливает обработку и удерживает поток в ожидании ответа пользователя. Узлы сгруппированы в семь окрашенных подграфов по функциональным фазам: жёлтый — маршрутизация и контроль тарифа; оранжевый — обработка дополнительных вопросов; фиолетовый — точечная правка готового договора; зелёный — классификация сделки; синий — проверка общих норм и поиск специальных; светло-жёлтый — проверки формы и достаточности данных; розовый — генерация и итеративная валидация; бирюзовый — формирование рекомендаций и сохранение результата. Сплошные стрелки изображают безусловные и условные переходы между узлами, подписи на стрелках раскрывают предикат соответствующего перехода. Пунктирные стрелки оранжево-красного цвета выделены отдельно и обозначают переходы, прерывающие обработку графа для диалогового уточнения у пользователя — ответ пользователя возобновляет обработку в соответствующем узле проверки. Пунктирная циклическая стрелка из узла `followup_subagent_search` в самого себя обозначает итеративный цикл изолированного поиска по корпусу до достижения целевого ответа или исчерпания лимита попыток.

